GLP: A Grassroots, Multiagent, Concurrent, Logic Programming Language for AI (Full Version)

Ehud Shapiro
London School of Economics and Weizmann Institute of Science

A grassroots platform is a multiagent distributed system in which multiple independent instances can form and operate independently of each other and of any global resource, yet may coalesce into ever larger instances, possibly resulting in a single global instance. Grassroots platforms aim to offer an egalitarian/democratic alternative to centralised/autocratic and decentralised/plutocratic global platforms.

Here, we present Grassroots Logic Programs (GLP), a multiagent concurrent logic programming language designed for the implementation of grassroots platforms: we recall the standard operational semantics of logic programs; introduce the concurrent operational semantics of GLP as its restriction; recall multiagent atomic transactions; use them to introduce a multiagent operational semantics of GLP; and prove multiagent GLP to be grassroots.

These mathematical foundations are being used by AI to implement GLP as well as to program in GLP: a workstation-based implementation of concurrent GLP in Dart was derived from the concurrent operational semantics of GLP; a multiagent smartphone-based implementation of GLP in Dart/Flutter is being developed based on the multiagent operational semantics of GLP; a moded type system for GLP was designed to facilitate collaborative human-AI development of GLP programs; GLP implementations of grassroots platforms for the social graph, social networks, currencies and bonds have been derived by AI from mathematical specifications.

While concurrent logic programming and its powerful programming techniques have been known for four decades, adoption has been hampered by their inaccessibility to the average programmer. With AI as a hyper-programmer, this limitation is removed and the abstract nature and expressive power of concurrent logic programming can be put into effective use.

Keywords: concurrent logic programming, grassroots platforms, operational semantics, multiagent transition systems, AI programming

1 Introduction

Grassroots. Grassroots platforms aim to offer an egalitarian and democratic alternative to centralised and autocratic (Facebook) and decentralised and plutocratic (Bitcoin) global platforms [45]. A digital platform is *grassroots* [43, 47] if it can have multiple instances that can (*i*) operate independently of each other and of any global resource other than the network, and (*ii*) coalesce into ever larger instances, possibly resulting in a single global instance. A grassroots platform aims to operate solely on the smartphones of its participants.

Specifications of grassroots platforms that have been provided and proven grassroots include the grassroots social graph [44, 47] — the infrastructure platform on which all others build — grassroots social networks [44], grassroots cryptocurrencies and bonds [42, 22, 46], and grassroots federations [58, 14]; the unifying formal framework being volitional multiagent atomic transactions [24]. Working GLP implementations of the social graph (this paper), child-safe social networks, grassroots cryptocurrencies and bonds [46], and secure recovery from major faults [8] have been derived by AI from these specifications. The Scuttlebutt protocol and social network [19] is perhaps the sole example of a deployed grassroots platform to date.

To appear in EPTCS.

© Shapiro
This work is licensed under the
Creative Commons Attribution License.

Grassroots Logic Programs. Here, we present Grassroots Logic Programs (GLP), a multiagent, concurrent, logic programming language, designed for the implementation of grassroots platforms. Syntactically, GLP extends Logic Programs (LP) [27] with:

1. **Readers:** Each logic variable X (now referred to as *writer*) is paired with a *reader* $X?$, which is assigned a value only once its paired writer X is assigned that value;
2. **Single-Occurrence (SO):** A variable may occur at most once in a goal or a clause; and
3. **Single-Reader Single-Writer (SRSW):** A writer occurs in a clause iff its paired reader does.

The result eschews unification for simple term matching, and conjures both linear logic [12] and futures/promises [3, 9]: An assignment to a variable may be produced at most once, via the sole occurrence of a writer (promise), and consumed at most once, via the sole occurrence of its paired reader (future).

Hence, in a multiagent distributed implementation, if a writer and its paired reader are held by different agents, an assignment to the writer is realised as a single message from the writer-holding agent to the reader-holding agent. The source of all the powerful concurrent logic programming techniques is that such a message may, in turn, contain further readers and writers, enabling the concise expression of rich multidirectional communication modalities. In the simplest case, a reader sent in a message keeps the channel open for further messages from the sender (streaming); a writer sent in a message allows the receiver to reply. More generally, variables in a message paired with variables held by third agents can be used for arbitrary network reconfiguration (e.g. friend-mediated introduction, example below).

Semantics. The operational semantics of GLP is presented in two stages: First, we present the *concurrent operational semantics of GLP* (cGLP), an interleaving-based concurrent (single-agent) semantics defined as a restriction of standard LP semantics, preserving computation-as-deduction [20].

Second, we present multiagent transition systems and their specification via multiagent atomic transactions, and use them to define the *multiagent operational semantics of GLP* (maGLP), with (1) **Communication:** Named agents that operate independently while communicating to remote readers assignments made to paired local writers; (2) **Cold-calls:** A means for sending a term with variables to a named agent while retaining their paired variables locally. Cold-calls allow two agents in disconnected components of the social graph to become the owners of paired logic variables. We illustrate maGLP via the grassroots social graph, in which agents may initiate friendships (bidirectional communication channels) via "cold-calls" as well as introduce mutual friends to each other. Lastly, we recall the definition of grassroots platforms and how to prove that a platform specified via atomic transactions is grassroots [43, 47, 24], and apply these to prove that maGLP is indeed grassroots.

Historical Context. The concurrent logic programming family emerged in the 1980s — Concurrent Prolog [35], GHC [61], PARLOG [6]; surveyed in [39] — and was simplified by *flattening* (restricting guards to primitive tests) in Flat Concurrent Prolog (FCP) [29] and Flat GHC; sequential FCP abstract machines [16] made commercial deployment feasible. Mode systems followed, including Ueda's moded Flat GHC [63, 64, 62], anticipating GLP's SO discipline as a type-system refinement. GLP can be understood as FCP with the SRSW restriction added, simplifying read-only unification [21]. These languages saw industrial deployment during the Japanese Fifth Generation project [30, 34], most prominently in the chat product Virtual Places, developed in FCP by Ubique [60] and acquired by AOL in 1995, after which the codebase was migrated to C. With the conclusion of the Fifth Generation project [55], concurrent logic programming went out of fashion. Its core ideas, however, persisted: pattern-matched message-passing among lightweight share-nothing processes, with single-assignment variables for synchronisation, found their most prominent industrial expression in Erlang [1] — perhaps the closest descendant of concurrent logic programming — and from there to Elixir and to actor frameworks such as Akka [26] in Scala and Orleans [28] in .Net. What was lost in this evolution was logic programming's metaprogram-

ming [32, 25] and the seamless integration of computation and synchronisation through paired logical variables; GLP recovers these, while maintaining the simplicity and efficiency of futures/promises.

AI. The foundations presented here are used by AI (Claude) in two distinct disciplines. *Implementing* GLP proceeds through three layers—a mathematical specification, an informal English-and-code specification derived from it by AI, and Dart code derived from that—with authority flowing math→spec→Dart but harmonised by back-and-forth. The concurrent and multiagent operational semantics are first refined into deterministic counterparts, dGLP and madGLP [48]. From dGLP AI derived a workstation-based GLP implementation in Dart, and from madGLP a smartphone-based multiagent one in Dart/Flutter is being developed. Running these implementations surfaced defects, several at the mathematical level, that drove revisions to the operational semantics themselves. A moded type checker was implemented by AI in the same way [50]. *Programming* in GLP proceeds through three layers of its own—mathematical specifications, provided as guarded multiagent atomic transactions [24]; the type definitions and declarations agreed for the program together with the intended behaviour of each procedure; and the GLP code: the designer and AI first agree the types and intent, and only then does AI write, type-check, test, and debug the code, with the human providing design-level oversight rather than code-level intervention [50]. The type checker catches at compile time a characteristic class of mode errors—confusing a reader for its paired writer—otherwise manifesting as silent run-time suspensions; a befriending clause, for example, was rejected because a reader argument received a produced rather than a consumed value, and AI corrected it from the diagnostic alone. A secure GLP implementation using public-key cryptography, mutual attestations, and friend-based identity custodians is also being developed [8]. Example GLP programs in this paper are typed.

Paper outline. Section 2 presents GLP. Section 3 presents multiagent transition systems, the maGLP definition, and its safety properties. Section 4 presents the grassroots social graph. Section 5 proves maGLP is grassroots. Section 6 concludes. All proofs are deferred to the appendices.

2 GLP

We present GLP syntax and operational semantics and discuss its programming techniques.

2.1 Syntax

GLP extends LP by adding a paired *reader* $X?$ to every "ordinary" logic variable X, now called a *writer*.

Definition 2.1 (GLP Variables)**.** Let $\mathscr{V}$ denote the set of LP variables (identifiers beginning with uppercase), henceforth called *writers*. Define $\mathscr{V}? = \{X? \mid X \in \mathscr{V}\}$, called *readers*. The set of all GLP variables is $\hat{\mathscr{V}} = \mathscr{V} \cup \mathscr{V}?$. A writer X and its reader $X?$ form a *variable pair*.

GLP terms, unit goals, goals, and clauses are as in LP but defined over the variables in $\hat{\mathscr{V}}$.

Definition 2.2 (Single-Occurrence (SO) Invariant)**.** A term, goal, or clause satisfies the *single-occurrence (SO) invariant* if every variable occurs in it at most once.

Definition 2.3 (GLP Program, Goals)**.** A clause C satisfies the *single-reader/single-writer (SRSW) restriction* if it satisfies SO and a variable occurs in C iff its paired variable also occurs in C. A *GLP program* is a finite sequence of clauses satisfying SRSW; clauses for the same predicate form a *procedure*. The set of GLP goals $\hat{\mathscr{G}}(P)$ includes all goals over $\hat{\mathscr{V}}$ and the vocabulary of P that satisfy SO.

The purpose of the SRSW restriction is to prevent multiple writer occurrences racing to assign a variable; the simpler (SW) restriction is insufficient, since if a reader with multiple occurrences is assigned a term with a writer, the result is a writer with multiple occurrences. We use set notation also when referring to multisets.

Example 2.4 (Fair Merge)**.** Consider the quintessential concurrent logic program for fairly merging two streams, written in GLP:[1]

```
Stream ::= [] ;[_|Stream].

procedure merge(Stream?, Stream?, Stream).
merge([X|Xs], Ys, [X?|Zs?]) :- merge(Ys?, Xs?, Zs).
merge(Xs, [Y|Ys], [Y?|Zs?]) :- merge(Xs?, Ys?, Zs).
merge([], Ys, Ys?).
merge(Xs, [], Xs?).
```

and the goal `merge([1,2,3|Xs?],[a,b|Ys?],Zs)`, which satisfies SO; each clause satisfies SRSW. The first clause swaps inputs in the recursive call, ensuring fairness when both streams are available.

2.2 Operational Semantics

Definition 2.5 (Transition System, Computation, Run, Safe, Live, Correct (cf. [41, 24]))**.** A *transition system* is a tuple $TS = (C, c_0, T, \sim)$ where C is an arbitrary set of *configurations*, $c_0 \in C$ a designated *initial configuration*, $T \subseteq C \times C$ a set of *transitions*, each a pair $c \to c'$ of non-identical configurations $c \neq c' \in C$, and $\sim$ a *partial equivalence relation* on T, symmetric and transitive; its domain $\{t \in T : t \sim t\}$ is partitioned into countably many *classes*, $[t]$ being the class of t, and a transition outside the domain belongs to no class.

A *computation* is a (nonempty, finite or infinite) sequence of configurations $c_1, c_2, \ldots$; it is a *run* if $c_1 = c_0$, and *safe* if $c_i \to c_{i+1} \in T$ for every two consecutive configurations. We write $c \xrightarrow{*} c'$ for the existence of a safe computation from c to c' (empty if $c = c'$). A class $[t] \in T/\sim$ is *enabled* in c if $c \to c' \in [t]$ for some c'. A run is *live* if no class $[t]$ is enabled in every configuration of some suffix in which no member of $[t]$ occurs, and *correct* if it is safe and live.

A transition in no class may occur in a correct run and is never required to [24]. The $\sim$ of cGLP below is an equivalence, every transition in a class; that of maGLP (Section 3) leaves the person's input outside its domain.

Definition 2.6 (Substitutions and Assignments)**.** A GLP *writer assignment* is a term of the form $X := T$, $X \in \mathscr{V}$, $T \notin \mathscr{V}$, satisfying SO. Similarly, a GLP *reader assignment* is a term of the form $X? := T$, $X? \in \mathscr{V}?$, $T \notin \mathscr{V}$, satisfying SO. A *writers (readers) substitution* σ is the substitution implied by a set of writer (reader) assignments that jointly satisfy SO. Given a writers assignment $X := T$, its *readers counterpart* is $X? := T$, and given a writers substitution σ, its *readers counterpart* $\sigma?$ is the readers substitution defined by $X?\sigma? = X\sigma$. Given a reader assignment $X? := T$, its *writers counterpart* is $X := T$, and given a readers substitution τ, its *writers counterpart* $\tau!$ is the writers substitution defined by $X\tau! = X?\tau$. The *pair completion* of a readers substitution σ is $\sigma^\star = \sigma \cup \sigma!$, applied to a fixed point.

Definition 2.7 (GLP Renaming, Renaming Apart)**.** A *GLP renaming* is an injective substitution $\rho : \hat{\mathscr{V}} \to \hat{\mathscr{V}}$ such that for each $X \in \mathscr{V}$: $X\rho \in \mathscr{V}$ and $X?\rho = (X\rho)?$. Two GLP terms *have a variable in common* if for some writer $X \in \mathscr{V}$, either X or $X?$ occurs in both. A GLP renaming σ *renames T' apart from T* if $T'\sigma$ and T have no variable in common.

Definition 2.8 (Writer MGU)**.** Given two GLP unit goals A and H, a *writer mgu* is a writers substitution σ such that $A\sigma = H\sigma$ and σ is most general among such substitutions.

[1] Moded-type definitions (`T ::= ...`) and declarations (`procedure ...`) used informally in examples are formally introduced in the Typed GLP companion paper [50].

Definition 2.9 (GLP Goal/Clause Reduction)**.** Given GLP unit goal A and clause C, with H :- B being the result of the GLP renaming of C apart from A, the *GLP reduction* of A with C *succeeds with result* (B,σ) if A and H have a writer mgu.

The cGLP transition equivalence below identifies "the same transition" across configurations as goals are instantiated by reader substitutions; following [24], we first assign each goal a persistent identity.

Definition 2.10 (Goal Identity)**.** Each unit goal in a cGLP computation carries a unique *identifier* assigned at spawn time: goals in the initial goal G_0 receive distinct initial identifiers, and goals spawned by a Reduce transition receive fresh identifiers. Communicate transitions preserve identifiers: the goal containing $X?$ retains its identity after instantiation.

Definition 2.11 (cGLP Transition System)**.** Given a GLP program P, an *asynchronous resolvent* over P is a pair (G,σ) where $G \in \hat{\mathscr{G}}(P)$ and σ is a readers substitution.

A transition system $cGLP = (\mathscr{C}, c_0, \mathscr{T}, \sim)$ is a *cGLP transition system* over P and initial goal G_0 satisfying SO if:

1. $\mathscr{C}$ is the set of all asynchronous resolvents over P
2. $c_0 = (G_0, \emptyset)$
3. $\mathscr{T}$ is the set of all transitions $(G,\sigma) \to (G',\sigma')$ satisfying either:
   (a) **Reduce:** there exists unit goal $A \in G$ such that $C \in P$ is the first clause for which the GLP reduction of A with C succeeds with result $(B,\hat{\sigma})$, $G' = (G \setminus \{A\}) \cup B(\hat{\sigma}?)^\star$, and $\sigma' = \sigma \circ \tau$, where τ is the restriction of $(\hat{\sigma}?)^\star$ to the readers of the writers of A
   (b) **Communicate:** $\{X? := T\} \in \sigma$, $X? \in G$, $G' = G\{X? := T\}$, and $\sigma' = \sigma$
4. $\sim$, the *cGLP transition equivalence*, relates $t_1 \sim t_2$ iff either both are Reduce transitions reducing the same goal (by identity, Definition 2.10) with the same clause, or both are Communicate transitions applying the counterpart of the same writer assignment to the same goal (by identity)

cGLP Reduce differs from LP in (1) the use of a writer mgu instead of a regular mgu and (2) the choice of the first applicable clause instead of any clause. The first is the fundamental use of GLP readers for communication and synchronisation. The second compromises on the or-nondeterminism of LP to allow writing fair concurrent programs, such as fair merge above. Note that or-nondeterminism is not completely eliminated, as different scheduling of arrival of assignments on the two input streams of `merge` may result in different orders in its output stream.

The cGLP Communicate rule realises the use of reader/writer pairs for asynchronous communication: it communicates an assignment from its writer to its paired reader.

Monotonicity. In LP, if a goal cannot be reduced, it will never be reduced. In cGLP, a goal that cannot be reduced now may be reduced in the future: if A and H have an mgu that writes on a reader $X? \in A$ with no writer mgu at present, another goal with X may reduce, assigning X, and later $X?$, to a value that will allow A and H to have a writer mgu. Conversely, in LP, if a goal A can be reduced now with some clause H :-B, with a regular mgu of A and H, it may not be reducible in the future due to variables that A shares with other goals being assigned values by reductions of other goals, preventing unification between the instantiated A and H. In cGLP, if a goal A can be reduced now (with a writers mgu), it can always be reduced in the future, as the SO invariant ensures that no other goal can assign any writer in A.

Implementation-wise, if a GLP goal A cannot be reduced now, but there is a readers substitution σ such that $A\sigma$ can be reduced, such readers are identified, the goal A *suspends* on these readers, and is rescheduled for another reduction attempt once any of them is assigned.

Despite these differences, cGLP has the same notion of logical consequence as LP.

Definition 2.12 (cGLP Proper Run, Outcome)**.** The *variables of a configuration* (G, σ) are those of G and of σ, its readers and the terms assigned to them. A cGLP run $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ is *proper* if for any $1 \leq i < n$, a variable of (G_{i+1}, σ_{i+1}) that is not one of (G_i, σ_i) is not one of any (G_j, σ_j), $j < i$; the *outcome* of a proper run is $(G_0 \ :\!-\ G_n)\sigma_n^\star$; and the run is *successful* if $G_n = \emptyset$.

Pair completion (Definition 2.6) is employed since σ_n accumulates only readers counterparts, leaving the writers of G_0 unbound. Let $/?$ be an operator that replaces every reader by its paired writer.

Proposition 2.13 (cGLP Computation is Deduction)**.** *Let* $(G_0 \ :\!-\ G_n)\sigma_n^\star$ *be the outcome of a proper cGLP run* $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$*. Then* $(G_0 \ :\!-\ G_n)\sigma_n^\star/?$ *is a logical consequence of* $P/?$*.*

We note two additional safety properties of cGLP runs.

Proposition 2.14 (SO Preservation)**.** *If the initial goal* G_0 *satisfies SO, then every goal in a proper cGLP run satisfies SO.*

Proposition 2.15 (Monotonicity)**.** *In any proper cGLP run, if unit goal A can reduce with clause C at step i, then either an instance of A has been reduced by step* $j > i$*, or an instance of A can still reduce with C at step* j*.*

Every cGLP run is safe, so correctness (Definition 2.5) reduces to liveness.

The SO invariant of GLP allows eschewing unification in favour of *term matching*: if two terms that jointly satisfy SO are unifiable, their mgu maps each variable in one term to a subterm of the other. Term matching thus performs joint term-tree traversal and collects variable assignments along the way; the detailed definition and table appear in Appendix B.

2.3 Guards

GLP clauses may include *guards*—tests that determine clause applicability.

Definition 2.16 (Guarded Clause)**.** A *guarded clause* has the form $H \ :\!-\ G \mid B$, where H is the head, G is a conjunction of guard predicates, and B is the body. The guard separator "`|`" is interpreted logically as a conjunction. The arguments of a guard are readers paired to head writers; the arguments of a *defined guard*, a guard predicate defined by a unit clause (Appendix D), may also be writers of the clause.

Guards have three-valued semantics. Each guard predicate explicitly defines its *success* condition. A guard *suspends* if it does not succeed but some instance of it under a readers substitution would succeed. A guard *fails* if no such instance exists. A guard conjunction succeeds if all members succeed; it suspends if any member suspends and none fail; it fails if any member fails.

Definition 2.9 of a GLP goal/clause reduction is augmented to succeed if the guard also succeeds.

Remark 2.17 (Guards and SRSW). Guard occurrences count toward SRSW satisfaction: if $X?$ occurs in a guard, its paired writer X must occur in the head and $X?$ may additionally occur once in the body; if X occurs in a defined guard, $X?$ occurs in the clause; if X occurs in a `widowed_writer` guard, $X?$ occurs in the head, X in the guard, and neither elsewhere in the clause; and if $X?$ occurs in a `widowed_reader` guard, X occurs in the head, $X?$ in the guard, and neither elsewhere in the clause.

Definition 2.18 (Ground Position, Ground-Guarded Writer)**.** A guard predicate g/n has a set $gr(g/n) \subseteq \{1, \ldots, n\}$ of *ground positions*: $i \in gr(g/n)$ iff the success of $g(T_1, \ldots, T_n)$ implies that T_i is ground. A writer X is *ground-guarded* in a guarded clause $H \ :\!-\ G \mid B$ if $X?$ occurs in T_i for some $g(T_1, \ldots, T_n) \in G$ and $i \in gr(g/n)$.

The ground positions are those the catalogue's Ground column marks (Appendix D).

Definition 2.19 (Ground Instantiation, SRSW of a Guarded Clause)**.** A *ground instantiation* of a guarded

clause C is $\theta \cup \theta?$, for a writers substitution θ whose domain is the set of ground-guarded writers of C and whose terms are ground. A guarded clause $C = H$ `:-` G `|` B *satisfies SRSW* if, for every ground instantiation $\bar{\theta}$ of C, $(H$ `:-` $B)\bar{\theta}$ satisfies SO and a variable occurs in $C\bar{\theta}$ iff its paired variable does, but for a variable occurring in a `widowed_writer` or `widowed_reader` guard of C, whose paired variable occurs in the head of C, and neither occurs elsewhere in C.

Definition 2.20 (GLP Reduction of a Guarded Clause)**.** Given a GLP unit goal A and a guarded clause C satisfying SRSW, with H `:-` G `|` B being the result of the GLP renaming of C apart from A, the *GLP reduction* of A with C *succeeds with result* $(B, \hat{\sigma} \cup \hat{\tau})$ if A and H have a writer mgu $\hat{\sigma}$ and $G(\hat{\sigma}?)^\star$ succeeds with $\hat{\tau}$, the union of the writer mgus of the term matchings of its defined guards, each with its unit clause renamed apart.

A defined guard assigns the writers of the clause among its arguments, and the writers in the values of its readers; cGLP Reduce (Definition 2.11) applies the former to the body and communicates the latter, as it does the writer mgu's.

Corollary 2.21 (SO Preservation with Guards)**.** *Proposition 2.14 holds for programs of guarded clauses satisfying SRSW.*

Remark 2.22 (Anonymous Variables). An *anonymous variable* is any variable whose name begins with `_` (e.g., `_`, `_Out`). Anonymous variables may appear anywhere a writer variable may appear; each occurrence denotes a fresh writer with no paired reader, the one exception to SRSW: an input a clause does not read is dropped, and what is assigned to it is discarded. Anonymous readers (`_?`, `_Name?`) are not permitted in a clause, in the head or the body: a reader whose writer is never written is a goal that can never be satisfied, so a clause writes every writer it holds, closing a stream it does not extend. The initial state of an agent (Section 3) is a goal, and the runtime writes its anonymous readers.

System predicates and body kernels. GLP *system predicates* such as `:=` (arithmetic assignment), `now` (clock access), `=..` (term composition/decomposition), and `random` (seeded random-number generation) are defined by GLP clauses whose bodies may invoke *body kernel predicates*—runtime-implemented primitives not directly accessible to user programs. The system predicate's own guards ensure that body kernel preconditions are met before invocation. For example, arithmetic assignment is defined recursively as follows:

```
Result? := N :- number(N?) | Result = N?.
Result? := X + Y :- number(X?), number(Y?) |
    '_add'(X?, Y?, Result).
Result? := X + Y :- otherwise |
    X1 := X?, Y1 := Y?, Result := X1? + Y1?.
```

The base case binds a plain number; the first addition clause guards that both operands are numbers and invokes the `'_add'` body kernel; the fallback clause recursively evaluates subexpressions. The full table of body kernels and the guard reference appear in Appendix D; the complete system-predicate definitions are in `programs/self.glp` of the repository.

2.4 GLP Programming Techniques

On the one hand, GLP has no unification, only pattern-matching and single-writer variables. On the other hand, goal reduction may write on several variables atomically. The result is that almost the entire range of concurrent logic programming techniques developed since the 1980s [39, 38] is available in GLP, including: streams and their fair, biased, and self-balancing merge [51, 52]; channels as generalised streams [59]; messages with reply variables and bounded buffers [57]; network reconfiguration and distributed programming [33]; object-oriented programming with mutable state via stream-recursion [53];

systems programming and computation control [37, 56]; and the full range of meta-interpreters — fail-safe, termination-detecting, tracing, and algorithmic debugging [32, 25]. Tested GLP renditions of these and further techniques — monitors, distributors, network switches, replicators, and the metainterpreter family — are in the `programs` directory of the public GLP repository.[2]

Two techniques require workarounds. The use of logic variables for mutual exclusion — standard in concurrent logic languages with atomic unification [39] — must be achieved in GLP via a monitor [15, 4], the standard distributed-computing solution. Similarly, general broadcast — distributing a value to multiple consumers — is not directly supported by GLP, since it would require multiple occurrences of the same reader, in violation of SO; it must be achieved by explicit *distributors*. We note that if moded-type checking can determine that the values to be distributed never contain writers, then the SO restriction on readers may safely be relaxed allowing broadcast natively; a future upgrade of the language. Excluded is the encoding of logic programs or-parallel by the and-parallelism of Concurrent Prolog [7, 40], which depends on general unification in a fundamental way.

3 Multiagent GLP

We extend GLP to multiple agents: Recall the notion of multiagent transition systems via multiagent atomic transactions [41, 47]; define multiagent GLP (maGLP) as a transactions-based multiagent transition system; and establish its safety properties.

3.1 Multiagent Transition Systems

We assume a countable set of *agents* Π, but consider only finite subsets of it, so when we refer to a particular set of agents $P \subset \Pi$ we assume P to be nonempty and finite. We use $\subset$ to denote the strict subset relation and $\subseteq$ when equality is also possible.

We use S^P to denote the set S indexed by the set P, and if $c \in S^P$ we use c_p to denote the member of c indexed by $p \in P$. Intuitively, think of such a $c \in S^P$ as an array of cells indexed by members of P with cell values in S.

Definition 3.1 (Local States, Configuration, Transaction, Participants)**.** Given agents $Q \subset \Pi$ and an arbitrary set S of *local states*, a *configuration* over Q and S is a member of $C := S^Q$. An *atomic transaction* over *participants* Q and states S is any pair of configurations $t = c \to c' \in C^2$ such that $c \neq c'$, with $t_p := c_p \to c'_p$ for any $p \in Q$.

Definition 3.2 (Multiagent Transition System)**.** Given agents $P \subset \Pi$ and an arbitrary set S of *local states* with a designated *initial local state* $s_0 \in S$, a *multiagent transition system* over P and S is a transition system $TS = (C, c_0, T, \sim)$ with *configurations* $C := S^P$, *initial configuration* $c_0 := \{s_0\}^P$, *transitions* $T \subseteq C^2$ being a set of transactions over P and S, and $\sim$ a partial equivalence on T.

Rather than specifying a multiagent transition system over a set of agents P directly, we specify it via atomic transactions.

Definition 3.3 (Transaction Closure)**.** Let $P \subset \Pi$, S a set of local states, and $C := S^P$; for any transition or transaction $t = c \to c'$, we say p is *stationary* in t if $c_p = c'_p$. For a transaction $t = (c \to c')$ over local states S with participants $Q \subseteq P$, the *P-closure of* t, $t{\uparrow}P$, is the set of transitions over P and S defined by:

$$t{\uparrow}P := \{t' \in C^2 : \forall q \in Q.(t_q = t'_q) \wedge \forall p \in P \setminus Q.(p \text{ is stationary in } t')\}$$

If R is a set of transactions, each $t \in R$ over some $Q \subseteq P$ and S, then the *P-closure of* R, $R{\uparrow}P$, is the set of

[2] `https://github.com/EShapiro2/GLP`

transitions over P and S defined by:

$$R{\uparrow}P := \bigcup_{t \in R} t{\uparrow}P$$

Given a partial equivalence $\sim$ on R, its *P-closure* $\sim{\uparrow}P$ is the relation on $R{\uparrow}P$ with $\hat{t}$ $(\sim{\uparrow}P)$ $\hat{t}'$ iff $\hat{t} \in t{\uparrow}P$ and $\hat{t}' \in t'{\uparrow}P$ for some $t \sim t'$.

Namely, the closure over $P \supseteq Q$ of a transaction t over Q includes all transitions t' over P in which members of Q do the same in t and in t', and the rest remain in their current (arbitrary) state. The closure likewise carries any partial equivalence on transactions to one on the induced transitions: where every participant of every transaction in R changes its state, distinct transactions in R have disjoint closures, and $\sim{\uparrow}P$ is a partial equivalence whenever $\sim$ is.

Definition 3.4 (Transactions-Based Multiagent Transition System)**.** Given agents $P \subset \Pi$, local states S with initial local state $s_0 \in S$, a set of transactions R, each $t \in R$ over some $Q \subseteq P$ and S, and a partial equivalence $\sim$ on R, the *transactions-based multiagent transition system* over P, S, R, and $\sim$ is the multiagent transition system $TS = (S^P, \{s_0\}^P, R{\uparrow}P, \sim{\uparrow}P)$.

3.2 From cGLP to maGLP

In extending GLP to multiple agents, each agent maintains its own asynchronous resolvent as its local state. The key insight is that GLP's variable pairs provide natural binary communication channels: when agent p assigns a writer X for which the paired reader $X?$ is held by agent q, the assignment $X := T$ must be communicated to q.

A key difference between cGLP and maGLP is in the initial state. In a multiagent transition system all agents must have the same initial local state s_0 (Definition 3.2), precluding the initial sharing of logic variables, as this would imply different initial states for different agents. We resolve this in two steps. First, we employ only anonymous logic variables "`_`" in the initial local states of agents: Anonymous variables are, on the one hand, syntactically identical, hence allow all initial states to be syntactically identical, and on the other hand represent unique variables, hence semantically all initial goals have unique, local, non-shared variables. The initial state of all agents is the atomic goal `agent(ch(_?,_),ch(_?,_))`, providing two bidirectional channels: the first to the person operating the machine, and the second to the network. The runtime writes its two anonymous readers: the person's responses to the first (the Input transaction below), and other agents' messages to the second (the Cold-call transaction below; Remark 2.22). The first channel is a bidirectional stream of GLP messages between the machine and the person, mediated by runtime UI support below the GLP layer and by physical UI hardware; messages flowing toward the person are rendered as UI elements, and the person's responses flow back as GLP messages. A person scripted in GLP is a program the runtime runs at the person's channel in the person's place, its writes being the Input transactions. With appropriate UI runtime support, a GLP message to the user may contain a writer, in which case the runtime presents it as a question and an eventual response from the person becomes the corresponding assignment. The second channel carries communication with other agents.

Second, the Cold-call transaction enables agents to bootstrap communication by establishing shared variables through the network infrastructure, realising the cold-call protocol for connecting previously-disconnected agents.

Definition 3.5 (Variables of an Agent)**.** Let $\mathcal{V}_p \subset \mathcal{V}$, for $p \in \Pi$, be pairwise disjoint infinite sets of writers. The *variables of agent p* are $\hat{\mathcal{V}}_p = \mathcal{V}_p \cup \{X? \mid X \in \mathcal{V}_p\}$.

Definition 3.6 (Multiagent GLP)**.** Given agents $P \subset \Pi$ and GLP program M, the *maGLP transition system* over P and M is the transactions-based multiagent transition system (Defini-

tion 3.4) over P, local states being asynchronous resolvents (G_p, σ_p) over M, initial local state $s_0 = (\{\texttt{agent(ch(_?,_),ch(_?,_))}\}, \emptyset)$, whose anonymous variables at agent p are variables of $\hat{\mathscr{V}}_p$, the following transactions $c \to c'$, and the equivalence $\sim$ given below:

1. **Reduce** p**:** A unary transaction with participant p where $c_p \to c'_p$ is a cGLP Reduce transition (Definition 2.11) whose GLP renaming maps the clause into $\hat{\mathscr{V}}_p$.
2. **Communicate** p **to** q**:** A transaction with participants $p, q \in P$ where $\{X? := T\} \in \sigma_p$, $X?$ occurs in G_q, and $c'_p = (G_p, \sigma_p \setminus \{X? := T\})$ and $c'_q = (G_q\{X? := T\}, \sigma_q)$ if $p \neq q$, and $c'_p = (G_p\{X? := T\}, \sigma_p \setminus \{X? := T\})$ if $p = q$.
3. **Cold-call** p **to** q**:** A binary transaction with participants $p \neq q \in P$ where the network output stream in c_p has a new message $\texttt{msg}(q, X)$, c'_p is the result of advancing the network output stream in c_p, and c'_q is the result of adding $\texttt{msg}(q, X)$ to the network input stream in c_q.
4. **Input** p**:** A unary transaction with participant p where $c'_p = (G_p, \sigma_p \cup \{X? := T\})$, $X?$ occurs in G_p, X is the person's — the writer of the tail of the person's input stream, or a writer in a message on the person's output stream in c_p — and T is a term whose variables are fresh variables of $\hat{\mathscr{V}}_p$.
5. $\sim$, the *maGLP transaction equivalence* on the transactions above, relates $t_1 \sim t_2$ iff both are Reduce transactions at the same agent p reducing the same goal (by identity, Definition 2.10) with the same clause, both are Communicate transactions from the same p to the same q applying the counterpart of the same writer assignment to the same goal (by identity), or both are Cold-call transactions from the same p to the same q delivering the same message; an Input transaction is related to none.

Note that Communicate may be unary or binary, depending on whether $p = q$. Every participant of every maGLP transaction changes its state, so $\sim\!\!\uparrow P$ is a partial equivalence (Section 3.1). Input is the person's act at their own agent, writing what the program handed them: the tail of the input stream, or a writer in a question the program put on the output stream. It belongs to no class, since the person is under no obligation to act, as a volition change is in [24]; what the person writes may name any agent, and this is how an agent learns of an agent it has not met. Communicate transfers assignments from writers to readers between agents sharing a paired reader and writer. Cold-call transfers a term with its variables to q through the network streams established in each agent's initial configuration, enabling the creation of paired variables among previously-disconnected agents. Cold-call is the exception, as once agents share a paired variable they can use it to communicate indefinitely; moreover, an agent with two friends (with which it shares channels) may introduce them to each other, also eschewing the need for a Cold-call.

3.3 Safety Properties of maGLP

The safety properties established for cGLP in Section 2 extend to maGLP. SO preservation (cf. Proposition 2.14) generalises directly:

Proposition 3.7 (maGLP SO Preservation)**.** *If the initial goals of all agents satisfy SO, then every goal in every agent's resolvent throughout a proper run satisfies SO.*

Definition 3.8 (maGLP Proper Run, Outcome)**.** An maGLP run is *proper* if the run at each agent is proper (Definition 2.12). The *outcome* of a proper maGLP run is the tuple of per-agent outcomes.

To relate maGLP to deduction, consider the *lifted system* L_r of a finite proper maGLP run r: the cGLP transition system whose resolvent is the union of all agents' local resolvents, whose initial goal includes a `network` goal with channels paired to each agent's network channels and, for each agent p, a $\texttt{person}_p$ goal with channels paired to p's person channel, and whose program M'_r is M augmented with the GLP definition of `network` and with a definition of each $\texttt{person}_p$ whose clauses write, in order, the terms the Input transactions of r at p write.

Proposition 3.9. *Every finite proper maGLP run r over P and M is simulated by a proper cGLP run of L_r whose outcome is a logical consequence of $M'_r/?$.*

Every maGLP run is safe, so correctness (Definition 2.5) reduces to liveness.

Implementation. The implementation-ready deterministic variants of GLP and maGLP (the latter deterministic at the agent, not system, level), along with formal correctness proofs that they implement their respective specifications, are presented in a companion paper [48].

4 The Grassroots Social Graph

The grassroots social graph is the foundation upon which all other grassroots platforms are built. Nodes represent cryptographically self-identified agents; edges represent authenticated bidirectional channels; connected components arise spontaneously through befriending.

Each agent processes messages from user and network input streams and maintains an outputs list with one typed entry per destination — `user_output`, `net_output`, and a `friend_output` per friend — on which the library routers `send_user`, `send_net`, and `send_friend` dispatch. The program supports four protocols: (1) *cold-call befriending*, where agents with no prior shared variables establish friendship by exchanging a response variable through the network; (2) *friend-mediated introduction*, where a mutual friend creates a channel pair and sends each half to the respective parties, establishing a direct connection; (3) *text messaging* between established friends via named output streams; and (4) *unfriending*, where either friend closes the shared channel.

The core of the program is the procedure `agent/4`; the key clauses follow. Module declarations (`exported/imported`, `#`-qualified calls) are elided from quoted clauses.

Channels. A *channel* is an authenticated bidirectional communication structure `ch(In, Out)`, where `In` is a stream of incoming messages and `Out` is a stream of outgoing messages. Two agents share a channel by holding opposite views of the same underlying stream pair: what one writes on its `Out`, the other reads from its `In`. The procedure `new_channel/2` creates such a pair:

```
new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).
```

A single call `new_channel(PQCh, QPCh)` binds `PQCh` to `ch(Xs?, Ys)` and `QPCh` to `ch(Ys?, Xs)`: the reader `Xs?` of one channel is paired with the writer `Xs` of the other (and likewise for `Ys/Ys?`), so each end's outgoing stream is the other end's incoming stream. Sending and receiving on a channel are:

```
send(X, ch(In, [X?|Out?]), ch(In?, Out)).
receive(X?, ch([X|In], Out?), ch(In?, Out)).
```

Channels are used in friend-mediated introduction (below) to establish a direct connection between two parties without a cold-call. They are also used by the cold-call protocol: when the recipient accepts, it returns a freshly-created setup-channel half as its response; the friend channel itself is conveyed over the setup channel by the shared commit procedure `befriend_commit`, completing the bidirectional link.

Cold-call befriending. The user's `connect(Target)` request triggers a cold-call: the agent sends `intro(Id, Resp)` on the network and injects the paired reader `Resp?` into its own user-input stream, so the eventual response surfaces locally as a normal user-side message:

```
agent(Id, [msg(person, Id1, connect(Target))|UserIn], NetIn, Outs) :-
    Id? =?= Id1?, ground(Target?) |
    send_net(msg(Target?, intro(Id?, Resp)), Outs?, Outs1),
    inject_msg(Resp?, Target?, Id?, UserIn?, UserIn1),
    agent(Id?, UserIn1?, NetIn?, Outs1?).
```

On the recipient side, an incoming `intro(From, Resp?)` captures the response writer `Resp`, forwarded to the user in a befriend prompt; the user's decision will assign it, transparently transported back to the originator:

```
agent(Id, UserIn, [msg(Id1, intro(From, Resp?))|NetIn], Outs) :-
    Id? =?= Id1? |
    send_user(msg(agent, person, befriend(From?, Resp)), Outs?, Outs1),
    agent(Id?, UserIn?, NetIn?, Outs1?).
```

When the user makes a decision (accept or reject), the agent processes the response:

```
agent(Id, [msg(person, Id1,
    decision(Dec, From, response(Resp?)))|UserIn], NetIn, Outs) :-
    Id? =?= Id1? |
    bind_response(Id?, Dec?, From?, Resp, Outs?, Outs1, NetIn?, NetIn1),
    agent(Id?, UserIn?, NetIn1?, Outs1?).
```

The helper `bind_response` handles both cases: on `yes`, it creates a fresh setup-channel pair via `new_channel`, assigns `accept(RetCh?)` to `Resp` (transported back to the initiator by the Communicate transaction) and commits the friendship over the local half via the shared `befriend_commit`, which conveys a fresh friend channel over the setup channel, registers its `friend_output` entry, and merges its incoming end into `NetIn`; on `no`, it informs the user the request was rejected. Both ends run the same idempotent commit. Its definition appears in Appendix F.

When Alice executes this protocol to befriend Bob, her agent sends `msg(bob, intro(alice, Resp))` on the `net_output` stream. The Cold-call transaction (Definition 3.6) transfers this message to Bob's network input stream, adding the writer `Resp` to Bob's resolvent. When Bob accepts, assigning `Resp` to `accept(RetCh?)` for a fresh setup channel, the Communicate transaction transfers this assignment back to Alice; `befriend_commit` then installs the friend channel at both ends.

Friend-mediated introduction. Friend-mediated introduction creates a fresh setup-channel pair via `new_channel/2` and sends one half to each of the introduced parties; on mutual acceptance a direct connection arises without a cold-call:

```
agent(Id, [msg(person, Id1, introduce(P, Q))|UserIn], NetIn, Outs) :-
    Id? =?= Id1?, ground(P?), ground(Q?), P? =?\= Q?,
    new_channel(PQCh, QPCh) |
    send_friend(P?, msg(Id?, P?, intro(Q?, QPCh?)), Outs?, Outs1),
    send_friend(Q?, msg(Id?, Q?, intro(P?, PQCh?)), Outs1?, Outs2),
    agent(Id?, UserIn?, NetIn?, Outs2?).
```

When Bob introduces Alice to Charlie, the setup-channel pair he creates contains readers that are transferred to Alice and Charlie via Communicate; each party accepts by acknowledging on the setup channel, and on mutual acknowledgement both commit the friendship via `befriend_commit` — a direct connection without any Cold-call.

Text messaging. Once two agents are friends, messages flow directly along their shared channel via the Communicate transaction, without cold-calls. The user's `send(Target, Text)` request is dispatched by `send_friend` on the `friend_output` entry registered under `Target` in `Outs`:

```
agent(Id, [msg(person, Id1, send(Target, Text))|UserIn], NetIn, Outs) :-
    Id? =?= Id1?, ground(Target?) |
    send_friend(Target?, msg(Id?, Target?, text(Text?)), Outs?, Outs1),
    agent(Id?, UserIn?, NetIn?, Outs1?).
```

On the other side, an incoming `text(Text)` from `From` is forwarded to the user as `received(From, Text)`:

```
agent(Id, UserIn, [msg(From, Id1, text(Text))|NetIn], Outs) :-
    Id? =?= Id1? |
    send_user(msg(agent, person, received(From?, Text?)), Outs?, Outs1),
    agent(Id?, UserIn?, NetIn?, Outs1?).
```

Unfriending. Either friend may unfriend the other, and the operation is one-way: no acknowledgement and no decline. The user's `unfriend(Target)` request notifies the peer on the friend channel, removes the `friend_output` entry, and tells the person the friend is gone:

```
agent(Id, [msg(person, Id1, unfriend(Target))|UserIn], NetIn, Outs) :-
    Id? =?= Id1?, ground(Target?) |
    send_friend(Target?, msg(Id?, Target?, unfriend), Outs?, Outs1),
    close_friend(Target?, Outs1?, Outs2),
    send_user(msg(agent, person, unfriended(Target?)), Outs2?, Outs3),
    agent(Id?, UserIn?, NetIn?, Outs3?).
```

The receiving side closes its own half on the incoming `unfriend`:

```
agent(Id, UserIn, [msg(From, Id1, unfriend)|NetIn], Outs) :-
    Id? =?= Id1?, ground(From?) |
    close_friend(From?, Outs?, Outs1),
    send_user(msg(agent, person, unfriended(From?)), Outs1?, Outs2),
    agent(Id?, UserIn?, NetIn?, Outs2?).
```

`close_friend` does nothing where the named party is not a friend, so both clauses are idempotent and an unfriend from both sides at once closes each half once.

Boot and deployment. Each agent runs on a separate isolate (later — separate smartphone). A boot clause reduces the universal initial state `agent(ch(_?,_),ch(_?,_))` (Definition 3.6) to the four-argument form used in the clauses above. The complete tested program — the type vocabulary, helper procedures, the UI mediator, the actors, the plays, and the boot variants — is the directory `programs/social/graph` of the public GLP repository (`https://github.com/EShapiro2/GLP`); a comprehensive Alice/Bob/Charlie scenario appears in Appendix F.

The social graph specified above, run as a maGLP program, is itself a grassroots platform: it inherits the grassroots property of maGLP (Corollary 5.4).

5 Multiagent GLP is Grassroots

This section establishes that maGLP is grassroots. We recall the notion of grassroots from [24]; the formal definitions appear in Appendix C.

Informally, a protocol is *grassroots* if two disjoint groups of agents can each operate independently—their interleaved correct runs are correct runs of the combined system—yet the combined system offers genuinely new behaviours that neither group could produce on its own. The first requirement is *obliviousness*; the second is *interactivity*.

The Cold-call transaction (Definition 3.6) is the fundamental interactive transaction of maGLP. It allows an agent p in one group to connect to an agent q in another, establishing shared variables that span both groups—a behaviour that no interleaving of independent runs of the two groups could produce. An agent p cold-calls q once q is named in its state, and p learns of an agent it has not met from the person, by the Input transaction.

An agent is a constant of GLP. A group of agents running on its own is one whose members know no agent outside it.

Definition 5.1 (maGLP Protocol)**.** Let M be a GLP program and, for a finite $P \subset \Pi$, let $S(P)$ be the set of asynchronous resolvents over M in which no agent outside P occurs. The *maGLP protocol over M* assigns to each P the maGLP transition system over P and M (Definition 3.6) with its configurations restricted to $S(P)^P$.

Definition 5.2 (Names No Agent, Uses Cold Calls)**.** A GLP program *M names no agent* if no agent occurs in *M*. It *uses cold calls* if for every local state *s* of an agent *p* reachable in a run of the maGLP protocol over *M* and every agent *q*, some Input *p* from *s* writing a term in which *q* occurs, followed by Reduce *p* and Communicate *p* to *p* transactions, reaches a local state in which Cold-call *p* to *q* is enabled.

Theorem 5.3. *The maGLP protocol over a program that names no agent and uses cold calls is grassroots.*

Corollary 5.4. *The GLP implementation of the grassroots social graph (Section 4) is grassroots.*

6 Conclusion

While concurrent logic programming and its powerful programming techniques have been known for four decades [39], adoption has been hampered by their inaccessibility to the average programmer. Concurrent logic programming requires a substantial shift from sequential procedural or functional thinking to dataflow programming via partial bindings, with concurrent operational semantics in which communication occurs through binding paired logical variables which may contain further logic variables. Our experience to date is that AI is highly effective and productive at programming from mathematical specifications when the target is a typed high-level language such as GLP, where types constrain the space of legal programs and provide a checkable specification at the human-AI interface [50]. Moreover, our preliminary experience suggests that the abstract nature of concurrent logic programming renders it a more powerful and productive language than mainstream languages for collaborative human-AI abstract specifications-based program development.

Constitutional governance of grassroots communities and federations builds on Constitutional Consensus [18, 23] and digital social contracts [5, 54], both realisable in GLP. The GLP runtime and example programs are open-source at `https://github.com/EShapiro2/GLP`.

References

[1] Joe Armstrong (2010): *Erlang*. *Communications of the ACM* 53(9), pp. 68–75, doi:10.1145/1810891.1810910.

[2] Dan Bachar & Ehud Shapiro (2026): *GLP Networking API Specification*. ArXiv, to appear.

[3] Henry G. Baker & Carl Hewitt (1977): *The Incremental Garbage Collection of Processes*. In: *Proceedings of the 1977 Symposium on Artificial Intelligence and Programming Languages*, ACM, pp. 55–59, doi:10.1145/800228.806932.

[4] Per Brinch Hansen (1973): *Operating System Principles*. Prentice-Hall.

[5] Luca Cardelli, Liav Orgad, Gal Shahaf, Ehud Shapiro & Nimrod Talmon (2020): *Digital social contracts: A foundation for an egalitarian and just digital society*. In: *CEUR Proceedings of the First International Forum on Digital and Democracy*, 2781, CEUR-WS, pp. 51–60. Available at `https://ceur-ws.org/Vol-2781/paper5.pdf`.

[6] Keith Clark & Steve Gregory (1986): *PARLOG: parallel programming in logic*. *ACM Transactions on Programming Languages and Systems (TOPLAS)* 8(1), pp. 1–49, doi:10.1145/5001.5390.

[7] Michael Codish & Ehud Shapiro (1986): *Compiling OR-parallelism into AND-parallelism*. In: *International Conference on Logic Programming*, Springer, pp. 283–297, doi:10.1007/3-540-16492-8_82.

[8] Ohad Eitan, Idit Keidar & Ehud Shapiro (2026): *Securing People and their Machines Against Major Faults (Full Version)*. *Proceedings of The 28th International Symposium on Stabilization, Safety, and Security of Distributed Systems (SSS 2026), to appear.* Available at `https://arxiv.org/abs/2607.02304`. ArXiv:2607.02304.

[9] Daniel P. Friedman & David S. Wise (1976): *The Impact of Applicative Programming on Multiprocessing*. *Indiana University Computer Science Department Technical Report* (TR-26).

[10] Haim Gaifman & Ehud Shapiro (1989): *Fully abstract compositional semantics for logic programs*. In: *Proceedings of the 16th ACM SIGPLAN-SIGACT symposium on Principles of programming languages*, pp. 134–142.

[11] Daniel T. Gillespie (1977): *Exact stochastic simulation of coupled chemical reactions*. *The Journal of Physical Chemistry* 81(25), pp. 2340–2361, doi:10.1021/j100540a008.

[12] Jean-Yves Girard (1987): *Linear Logic*. *Theoretical Computer Science* 50(1), pp. 1–101, doi:10.1016/0304-3975(87)90045-4.

[13] James Golike, Andy Lewis-Pye & Ehud Shapiro (2026): *Grassroots Social Contracts: From Legal Text to an App (Full Version)*. *arXiv:2609.06231*.

[14] Daniel Halpern, Ariel D Procaccia, Ehud Shapiro & Nimrod Talmon (2024): *Federated Assemblies*. *Proc AAAI 2025; arXiv preprint arXiv:2405.19129*. Available at `https://arxiv.org/abs/2405.19129`.

[15] C. A. R. Hoare (1974): *Monitors: An operating system structuring concept*. *Communications of the ACM* 17(10), pp. 549–557, doi:10.1145/355620.361161.

[16] Avshalom Houri & Ehud Shapiro (1989): *A sequential abstract machine for Flat Concurrent Prolog*. *The Journal of Logic Programming* 7(2), pp. 85–123, doi:10.1016/0743-1066(89)90011-3.

[17] Guy L. Steele Jr., Doug Lea & Christine H. Flood (2014): *Fast Splittable Pseudorandom Number Generators*. In: *Proceedings of the 2014 ACM International Conference on Object Oriented Programming Systems Languages & Applications (OOPSLA 2014)*, ACM, pp. 453–472, doi:10.1145/2660193.2660195.

[18] Idit Keidar, Andrew Lewis-Pye & Ehud Shapiro (2025): *Constitutional Consensus*. *arXiv preprint arXiv:2505.19216*. Available at `https://arxiv.org/abs/2505.19216`.

[19] Anne-Marie Kermarrec, Erick Lavoie & Christian Tschudin (2020): *Gossiping with append-only logs in secure-Scuttlebutt*. In: *Proceedings of the 1st international workshop on distributed infrastructure for common good*, pp. 19–24, doi:10.1145/3428662.3428794.

[20] Robert Kowalski (1974): *Predicate logic as programming language*. In: *IFIP congress*, 74, pp. 569–574.

[21] Giorgio Levi & Catuscia Palamidessi (1985): *The Semantics of the Read-Only Variable*. In: *Proc. Symposium on Logic Programming*, IEEE, pp. 128–137.

[22] Andrew Lewis-Pye, Oded Naor & Ehud Shapiro (2023): *Grassroots Flash: A Payment System for Grassroots Cryptocurrencies*. *arXiv preprint arXiv:2309.13191*. Available at `https://arxiv.org/abs/2309.13191`.

[23] Andrew Lewis-Pye & Ehud Shapiro (2025): *Morpheus Consensus: Excelling on trails and autobahns*. *Proc. 29th Conference on Principles of Distributed Systems, OPODIS 2025. arXiv preprint arXiv:2502.08465*, doi:10.4230/LIPIcs.OPODIS.2025.35.

[24] Andy Lewis-Pye & Ehud Shapiro (2026): *Volition-Guarded Multiagent Atomic Transactions: Describing People and their Machines*. *arXiv preprint arXiv:2604.25596*. Available at `https://arxiv.org/abs/2604.25596`.

[25] Yossi Lichtenstein & Ehud Shapiro (1988): *Concurrent algorithmic debugging*. *ACM SIGPLAN Notices* 24(1), pp. 248–260, doi:10.1145/68210.69239.

[26] Lightbend Inc. (2022): *Akka: Build Concurrent, Distributed, and Resilient Message-Driven Applications*. `https://akka.io`. Accessed October 2025.

[27] John W. Lloyd (1987): *Foundations of Logic Programming*, 2nd edition. Springer-Verlag, doi:10.1007/978-3-642-83189-8.

[28] Microsoft (2022): *Orleans: Cloud Native Application Framework*. `https://dotnet.github.io/orleans`. Accessed October 2025.

[29] Colin Mierowsky, Stephen Taylor, Ehud Shapiro, Jacob Levy & Shmuel Safra (1985): *The Design and Implementation of Flat Concurrent Prolog*. Technical Report CS85-09, Department of Computer Science, The Weizmann Institute of Science, Rehovot, Israel.

[30] Tohru Moto-Oka (1983): *Overview to the fifth generation computer system project*. In: *Proceedings of the 10th annual international symposium on Computer architecture*, pp. 417–422.

[31] Corrado Priami, Aviv Regev, Ehud Shapiro & William Silverman (2001): *Application of a stochastic name-passing calculus to representation and simulation of molecular processes*. *Information processing letters* 80(1), pp. 25–31, doi:10.1016/s0020-0190(01)00214-9.

[32] Shmuel Safra & Ehud Shapiro (1987): *Meta-interpreters for real*. In: *Concurrent Prolog: Collected Papers*, chapter 25, 2, MIT Press, Cambridge, MA, pp. 166–179. Also in Information Processing 86, pp. 271–278, North-Holland, 1986.

[33] A. Shafrir & E. Shapiro (1987): *Distributed Programming in Concurrent Prolog*. In Ehud Shapiro, editor: *Concurrent Prolog: Collected Papers, Volume 1*, MIT Press, Cambridge, MA, pp. 318–338.

[34] Ehud Shapiro (1983): *The fifth generation project—a trip report*. *Communications of the ACM* 26(9), pp. 637–641, doi:10.1145/358172.358179.

[35] Ehud Shapiro (1983): *A subset of Concurrent Prolog and its interpreter*. *ICOT Technical Report, TR-003*.

[36] Ehud Shapiro (1984): *Alternation and the computational complexity of logic programs*. *The Journal of Logic Programming* 1(1), pp. 19–33.

[37] Ehud Shapiro (1984): *Systems programming in concurrent prolog*. In: *Proceedings of the 11th ACM SIGACT-SIGPLAN symposium on Principles of Programming Languages*, pp. 93–105, doi:10.1145/800017.800520.

[38] Ehud Shapiro (1987): *Concurrent Prolog: collected papers (Vols. I and II)*. MIT press. Available at `https://weizmann.elsevierpure.com/en/publications/concurrent-prolog-collected-papers/`.

[39] Ehud Shapiro (1989): *The family of concurrent logic programming languages*. *ACM Computing Surveys (CSUR)* 21(3), pp. 413–510, doi:10.1145/72551.72555.

[40] Ehud Shapiro (1989): *Or-parallel prolog in flat concurrent prolog*. *The Journal of Logic Programming* 6(3), pp. 243–267, doi:10.1016/0743-1066(89)90016-2.

[41] Ehud Shapiro (2021): *Multiagent Transition Systems for Composing Fault-Resilient Protocol Stacks*. *arXiv preprint arXiv:2112.13650*. Available at `https://arxiv.org/abs/2112.13650`.

[42] Ehud Shapiro (2022): *Grassroots Currencies: Turning Mutual Trust into Liquidity*. *arXiv preprint arXiv:2202.05619*. Available at `https://arxiv.org/abs/2202.05619`.

[43] Ehud Shapiro (2023): *Grassroots Distributed Systems: Concept, Examples, Implementation and Applications (Brief Announcement)*. In: *37th International Symposium on Distributed Computing (DISC 2023). (Extended version: arXiv:2301.04391)*, LIPICS, Italy, pp. 47:1–47:7. Available at `https://arxiv.org/abs/2301.04391`.

[44] Ehud Shapiro (2023): *Grassroots Social Networking: Serverless, Permissionless Protocols for Twitter/LinkedIn/WhatsApp*. In: *OASIS '23*, Association for Computing Machinery, doi:10.1145/3599696.3612898.

[45] Ehud Shapiro (2025): *Characterising Global Platforms: Centralised, Decentralised, Federated, and Grassroots*. *arXiv preprint arXiv:2511.03286*. Available at `https://arxiv.org/abs/2511.03286`.

[46] Ehud Shapiro (2026): *Grassroots Bonds: Financing by the People, for the People*. arXiv:2603.13671. arXiv:2603.13671.

[47] Ehud Shapiro (2026): *Grassroots Platforms with Atomic Transactions: Social Graphs, Cryptocurrencies, and Democratic Federations*. In: *Proceedings of the 27th International Conference on Distributed Computing and Networking*, pp. 71–81, doi:10.1145/3772290.3772309. ArXiv preprint arXiv:2502.11299.

[48] Ehud Shapiro (2026): *Implementing Grassroots Logic Programs with Multiagent Transition Systems and AI (Full Version)*. *arXiv:2602.06934. Summary to appear in Proc. of LOPSTR+PPDP'26*. Available at `https://arxiv.org/abs/2602.06934`.

[49] Ehud Shapiro (2026): *sGLP: Simulating People with Stochastic Agents*. arXiv:2610.14738. arXiv:2610.14738.

[50] Ehud Shapiro (2026): *Types for Grassroots Logic Programs (Full version)*. arXiv preprint arXiv:2601.17957. Available at `https://arxiv.org/abs/2601.17957`.

[51] Ehud Shapiro & Colin Mierowsky (1984): *Fair, biased, and self-balancing merge operators: Their specification and implementation in Concurrent Prolog*. New Generation Computing 2(3), pp. 221–240, doi:10.1007/BF03037058.

[52] Ehud Shapiro & Shmuel Safra (1986): *Multiway merge with constant delay in Concurrent Prolog*. New Generation Computing 4(2), pp. 211–216, doi:10.1007/BF03037442.

[53] Ehud Shapiro & Akikazu Takeuchi (1983): *Object oriented programming in Concurrent Prolog*. New Generation Computing 1(1), pp. 25–48, doi:10.1007/BF03037020.

[54] Ehud Shapiro & Nimrod Talmon (2022): *Foundations for Grassroots Democratic Metaverse*. In: Proceedings of the 21st International Conference on Autonomous Agents and Multiagent Systems, AAMAS '22, International Foundation for Autonomous Agents and Multiagent Systems, Richland, SC, pp. 1814–1818, doi:10.65109/CNAL9987.

[55] Ehud Shapiro & David HD Warren (1993): *The 5th Generation Project: personal perspectives*. Communications of the ACM 36(3), pp. 47–49.

[56] William Silverman, Michael Hirsch, Avshalom Houri & Ehud Shapiro (1988): *The Logix system user manual Version 1.21*. In: Concurrent Prolog: Collected Papers, MIT Press, pp. 46–77.

[57] A. Takeuchi & K. Furukawa (1987): *Bounded Buffer Communication in Concurrent Prolog*. In Ehud Shapiro, editor: Concurrent Prolog: Collected Papers, Volume 1, MIT Press, Cambridge, MA, pp. 464–475.

[58] Nimrod Talmon & Ehud Shapiro (2025): *Grassroots Federation: Fair Democratic Governance at Scale*. arXiv preprint arXiv:2505.02208; also Proc. of AAMAS'26. Available at `https://arxiv.org/abs/2505.02208`.

[59] E. D. Tribble, M. S. Miller, K. Kahn, D. G. Bobrow, C. Abbott & E. Shapiro (1987): *Channels: A Generalization of Streams*. In Ehud Shapiro, editor: Concurrent Prolog: Collected Papers, Volume 1, MIT Press, Cambridge, MA, pp. 446–463.

[60] Ubique (1994): *Ubique. Wikipedia, The Free Encyclopedia*. Available at `https://en.wikipedia.org/wiki/Ubique_(company)`.

[61] Kazunori Ueda (1986): *Guarded Horn Clauses*. In: Logic Programming '85, Lecture Notes in Computer Science 221, Springer, pp. 168–179, doi:10.1007/3-540-16479-0_17.

[62] Kazunori Ueda (2001): *Resource-passing concurrent programming*. Proceedings of TACS 2001, pp. 95–126, doi:10.1007/3-540-45500-0_5.

[63] Kazunori Ueda & Masao Morita (1994): *Moded Flat GHC and Its Message-Oriented Implementation Technique*. New Generation Computing 12(4), pp. 337–368, doi:10.1007/BF03038307.

[64] Kazunori Ueda & Masao Morita (1995): *I/O mode analysis in concurrent logic programming*. In: Proceedings of the International Symposium on Theory and Practice of Parallel Programming, Springer, pp. 356–368, doi:10.1007/BFb0026579.

A Logic Programs

We recall standard Logic Programs (LP) notions of syntax, most-general unifier (mgu), and semantics via goal reduction.

Definition A.1 (Logic Programs Syntax)**.** We employ standard LP notions. Let $\mathscr{V}$ denote the set of *variables* (identifiers beginning with uppercase). A *term* is a variable, a constant (numbers, strings, or the empty list `[]`), or a compound term $f(T_1,\ldots,T_n)$ with functor f and subterms T_i. Let $\mathscr{T}$ denote the set of all terms. We use standard list notation: `[X|Xs]` for a list cell, `[X1,...,Xn]` for finite lists. A term is *ground* if it contains no variables.

A *unit goal* is a compound term or a string, also commonly referred to as an *atom*. A *goal* is a multiset of unit goals; the empty goal is written `true`. A *clause* A `:-` B has head A (a unit goal) and body B (a goal); a *unit clause* has empty body. A *logic program* is a finite set of clauses; clauses for the same predicate form a *procedure*. Let $\mathscr{G}(P)$ denote the set of goals over the vocabulary of the program P.

A *substitution* σ is an idempotent function $\sigma : \mathscr{V} \to \mathscr{T}$, a mapping from variables to terms applied to a fixed point. By convention, $\sigma(x) = x\sigma$. Let Σ denote the set of all substitutions. We assume standard notions of instance, ground, renaming, renaming apart, unifier, and most-general unifier (mgu).

Definition A.2 (LP Goal/Clause Reduction)**.** Given an LP unit goal A and clause C, with H `:-` B being the result of renaming C apart from A, the *LP reduction* of A with C *succeeds with* (B, σ) if A and H have an mgu σ.

Definition A.3 (Logic Programs Transition System)**.** A transition system $LP(P) = (C, c_0, T)$ is a *Logic Programs transition system* for a logic program P and initial goal $G_0 \in \mathscr{G}(P)$, if $C = \mathscr{G}(P) \times \Sigma$, $c_0 = (G_0, \emptyset)$, and T is the set of all transitions $(G, \sigma) \to (G', \sigma')$ such that for some unit goal $A \in G$ and clause $C \in P$ the LP reduction of A with C succeeds with $(B, \hat{\sigma})$, $G' = (G \setminus \{A\} \cup B)\hat{\sigma}$, and $\sigma' = \sigma \circ \hat{\sigma}$.

LP has two forms of nondeterminism: the choice of $A \in G$, called *and-nondeterminism*, and the choice of $C \in P$, called *or-nondeterminism*, and as such are closely-related to Alternating Turing Machines [36].

Definition A.4 (Proper and Successful Run, Outcome)**.** A run $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$ of $LP(P)$ is *proper* if for any $1 \leq i < n$, a variable that occurs in G_{i+1} but not in G_i also does not occur in any G_j, $j < i$. If proper, the *outcome* of ρ is $(G_0$ `:-` $G_n)\sigma_n$. Such a run is *successful* if $G_n = \emptyset$.

The following proposition justifies the computation-as-deduction view of LP [20], calling a proper LP run a *derivation* and a successful proper run a *successful derivation*.

Proposition A.5 (LP Computation is Deduction)**.** *The outcome* $(G_0$ `:-` $G_n)\sigma$ *of a proper run of* $LP(P)$ *is a logical consequence of* P*.*

The $LP(P)$ transition system allows defining several denotational semantic notions for a program P: (1) the *clause semantics* is the set of all outcomes of all proper runs with an initial most-general unit goal (arguments are distinct variables), closely related to the fully-abstract compositional semantics of LP [10]; (2) the *atom semantics* is the set of all outcomes of all successful derivations with an initial most-general unit goal; (3) the *ground atom semantics* is the standard model-theoretic semantics, the set of ground instances of the atom semantics over the Herbrand universe of P [27].

B Term Matching

If two terms T_1 and T_2 that jointly satisfy SO are unifiable with an mgu σ, then σ maps any variable in T_1 to a subterm of T_2 and vice versa. Hence, the SO invariant of GLP allows eschewing unification in favour of *term matching* that performs joint term-tree traversal and collects variable assignments along the way.

Definition B.1 (Term Matching)**.** Given two terms T_1 and T_2 that jointly satisfy SO, their *term matching* proceeds via the joint traversal of the term-trees of T_1 and T_2, consulting the following table at each pair of joint vertices, where X_1, X_2 denote writers, $X_1?, X_2?$ denote readers, and f/n denotes a non-variable term, a constant when $n = 0$ and a compound term when $n > 0$:

$T_1 \backslash T_2$	Writer X_2	Reader $X_2?$	Term f_2/n_2
Writer X_1	fail	$X_1 := X_2?$	$X_1 := T_2$
Reader $X_1?$	$X_2 := X_1?$	fail	suspend on $X_1?$
Term f_1/n_1	$X_2 := T_1$	fail	fail if $f_1 \neq f_2$ or $n_1 \neq n_2$

The writer mgu is the union of all writer assignments if no *fail* was encountered and the suspension set is empty.

Remark B.2. In an actual implementation, assuming T_1 is a goal term and T_2 a head term, the case of $X_1?$ and T_2 would add $X_1?$ to the set of readers the goal would suspend upon.

C Grassroots Definitions

This appendix recalls from [24] the definitions of protocols and of the grassroots property referenced in Section 5, and the condition under which a protocol defined by transactions is oblivious, and extends the latter to a protocol in which the person acts.

C.1 Protocols and the Grassroots Property

A protocol is a family of multiagent transition systems, one for each finite set of agents $P \subset \Pi$, sharing an underlying set of local states with a designated initial state.

Definition C.1 (Local-States Function [24])**.** A *local-states function* $S : 2^{\Pi} \to 2^{\mathscr{S}}$ maps every finite set of agents $P \subset \Pi$ to a set of local states $S(P) \subset \mathscr{S}$ that includes s_0 and satisfies $P \subset P' \implies S(P) \subset S(P')$.

Given a local-states function S, we use $C(P) := S(P)^P$ for configurations over P and $c_0(P) := \{s_0\}^P$ for the initial configuration.

Definition C.2 (Protocol [24])**.** A *protocol* $\mathscr{F}$ over a local-states function S is a family of multiagent transition systems with exactly one transition system $\mathscr{F}(P) = (C(P), c_0(P), T(P), \sim(P))$ for every finite $P \subset \Pi$.

A *correct* run of a transition system is safe and live (Definition 2.5); the runs here are safe by construction, so correctness coincides with liveness.

Lemma C.3 (Extension [24])**.** *Every finite safe run of a transition system is a prefix of a correct run of it.*

Informally, in a grassroots protocol two disjoint groups of agents can each operate independently—their interleaved correct runs are correct runs of the combined system—yet the combined system offers genuinely new behaviours that neither group could produce on its own. To define this formally, we first define the interleaving of runs of two disjoint groups.

Definition C.4 (Interleaving [24])**.** Let $P, P' \subset \Pi$ be disjoint nonempty sets of agents, $r = c_0, c_1, \ldots$ a run of $\mathscr{F}(P)$, and $r' = d_0, d_1, \ldots$ a run of $\mathscr{F}(P')$. An *interleaving* of r and r' is a sequence $e_0, e_1, \ldots$ of configurations in $C(P \cup P')$ for which there exist non-decreasing sequences of indices $(i_k)_{k \geq 0}$ and $(j_k)_{k \geq 0}$ with $i_0 = j_0 = 0$ such that for every $k \geq 0$:

1. $(e_k)_p = (c_{i_k})_p$ for every $p \in P$,
2. $(e_k)_q = (d_{j_k})_q$ for every $q \in P'$,
3. if e_{k+1} exists, then exactly one of: (a) $i_{k+1} = i_k + 1$ and $j_{k+1} = j_k$ (a P-step), or (b) $i_{k+1} = i_k$ and $j_{k+1} = j_k + 1$ (a P'-step).

Moreover, if r is finite of length n then $i_k = n$ for some k, and if r is infinite then for every $m \geq 0$ there is a k with $i_k = m$; likewise for r' and (j_k).

Note that an interleaving is well-defined: since $P \subset P \cup P'$, the local-states function ensures $S(P) \subset S(P \cup P')$, and similarly for P', so each e_k is a valid configuration in $C(P \cup P')$. Also, $e_0 = c_0(P \cup P')$, since $(e_0)_p = (c_0)_p = s_0$ for all $p \in P$ and $(e_0)_q = (d_0)_q = s_0$ for all $q \in P'$.

Definition C.5 (Interaction, Interactive Run [24])**.** Let $Q \subset \Pi$. For a configuration c over Q and $R \subseteq Q$, write $c|_R$ for the restriction of c to R. A transition $e \to e'$ of $\mathscr{F}(Q)$ is an *interaction between* $x \neq y \in Q$ if $e|_{\{x\}} \neq e'|_{\{x\}}$ and $e|_{\{y\}} \neq e'|_{\{y\}}$, and $e|_{\{x\}} \to e'|_{\{x\}}$ is not a transition of $\mathscr{F}(\{x\})$ or $e|_{\{y\}} \to e'|_{\{y\}}$ is not a transition of $\mathscr{F}(\{y\})$. It is an *interaction between* disjoint nonempty $P, P' \subseteq Q$ if it is an interaction between some $x \in P$ and some $y \in P'$. A run $\hat{r}$ of $\mathscr{F}(Q)$ is *interactive between P and P'* if some transition of $\hat{r}$ is an interaction between them.

Definition C.6 (Oblivious, Interactive, Grassroots [24])**.** A protocol $\mathscr{F}$ is:

1. *oblivious* if for every disjoint nonempty $P, P' \subset \Pi$, every interleaving of a correct run of $\mathscr{F}(P)$ and a correct run of $\mathscr{F}(P')$ is a correct run of $\mathscr{F}(P \cup P')$.
2. *interactive* if for every $Q \subset \Pi$ and disjoint nonempty $P, P' \subseteq Q$, every prefix of a correct run of $\mathscr{F}(Q)$ containing no interaction between P and P' is a prefix of a correct run of $\mathscr{F}(Q)$ that contains one.
3. *grassroots* if it is oblivious and interactive.

Being oblivious means that two disjoint groups of agents, each running the protocol independently and correctly, do not interfere with each other: any interleaving of their independent correct runs is a correct run of the combined system. Being interactive means that two disjoint groups that have not interacted can still do so: the combined system has a correct continuation in which one transition engages an agent of each group. In an interleaving, each step changes the local states of agents in only one group; therefore, any transaction whose active participants span both groups yields a step that cannot occur in any interleaving. These definitions are the machine-layer specialisation, without guards, of the volitional grassroots framework of [24].

C.2 Transactions-Based Protocols

Definition C.7 (Transactions Over a Local-States Function [24])**.** Let S be a local-states function. A set of transactions R is *over* S if every transaction $t \in R$ is a transition over participants Q and local states $S(Q')$ for some $Q \subseteq Q' \subset \Pi$. Given such R and $P \subset \Pi$:

$$R(P) := \{t \in R : t \text{ has participants } Q \subseteq P \text{ and is over } S(Q') \text{ for some } Q' \subseteq P\}$$

A transaction of agents of P from local states of P may reach a local state of a larger set of agents, as when p adds to its state an agent outside P. Such a transaction is in $R(P \cup P')$ and not in $R(P)$, so an interleaving of correct runs of $\mathscr{F}(P)$ and $\mathscr{F}(P')$ may owe it a liveness obligation that the run of $\mathscr{F}(P)$ does not owe [24]. It is excluded as follows.

Definition C.8 (Contained [24])**.** A set of transactions R over a local-states function S is *contained* if for every $P \subset \Pi$, every $t = c \to c'$ in R with participants $Q \subseteq P$ and $c \in S(P)^Q$ is in $R(P)$.

Definition C.9 (Transactions-Based Protocol [24])**.** Let S be a local-states function and R a set of transactions over S, in which distinct transactions have disjoint P-closures for every $P \subset \Pi$, with a partial equivalence $\sim$. The *protocol $\mathscr{F}$ over R, S, and $\sim$* is defined by $\mathscr{F}(P) := (C(P), c_0(P), R(P)\!\uparrow\! P, {\sim}\!\uparrow\! P)$ for each $P \subset \Pi$, where $R(P)\!\uparrow\! P$ and $\sim\!\uparrow\! P$ are the P-closures (Definition 3.3) of $R(P)$ and of $\sim$ restricted to $R(P)$.

Since liveness applies to all classes, any class $[t]$ in $R(P \cup P')/\!\sim$ whose transactions have participants spanning both P and P' could obstruct obliviousness if enabled in an interleaving; so could a class whose transactions have participants in P and that is not a class of $\mathscr{F}(P)$, which containment excludes.

Proposition C.10 (Obliviousness Condition [24])**.** *A transactions-based protocol over a contained set of transactions is oblivious provided that for every disjoint nonempty $P,P' \subset \Pi$, no class whose transactions have participants spanning both P and P' is ever enabled in any interleaving of correct runs of $\mathscr{F}(P)$ and $\mathscr{F}(P')$.*

In maGLP the person acts: the Input transaction (Definition 3.6) is the person's, belongs to no class, and may bring into an agent's state the name of an agent outside its group. It is the machine-layer counterpart of a volition change in [24], and is added to a transactions-based protocol as follows.

Definition C.11 (Transactions-Based Protocol with Input)**.** Let $\mathscr{F}$ be the protocol over R, S, and $\sim$, and I a set of unary transactions over S, each $t \in I$ over one participant p and $S(Q')$ for some $Q' \ni p$. The *protocol over R, S, $\sim$ and input I* is $\mathscr{F}^I$ with $\mathscr{F}^I(P) := (C(P), c_0(P), R(P)\!\uparrow\! P \cup I(P)\!\uparrow\! P, {\sim}\!\uparrow\! P)$ for each $P \subset \Pi$; a transition of $I(P)\!\uparrow\! P$ is in no class.

Proposition C.12 (Obliviousness Condition with Input)**.** *A transactions-based protocol with input over a contained set of transactions is oblivious provided that for every disjoint nonempty $P,P' \subset \Pi$, no class whose transactions have participants spanning both P and P' is ever enabled in any interleaving of correct runs of $\mathscr{F}^I(P)$ and $\mathscr{F}^I(P')$.*

Theorem C.13 (Transactions-Based Grassroots)**.** *A transactions-based protocol with input that satisfies the condition of Proposition C.12 and is interactive is grassroots.*

D Guards, Kernel and Seam Predicates

Guards, kernel predicates and seam predicates extend GLP programs with access to the GLP runtime state, operating system and hardware capabilities.

Predefined types. The predefined types, available in every GLP program, are `Stream(X)`, `List(X)`, `OpenStream(X)`, `DiffList(X)`, `Channel(In, Out)`, `Closed`, `MwmInput(X)`, `Number`, `Constant`, `Key`, `SignedTerm`, `Hash`, `Signature`, `GlobalName`, `NetMsg(X)`, `Answer`, `PeerAddress`, `App`, `Declaration`, `Removal`, `ProximityUnderlay`, `Network`, `TrustLevel`, `PlaceEvent`, `Presence`, `Reach`, `Token`, `ClockStep`, `Compiled`, `Content`, `Path`, `Input(T)`, `Widget`, `Draw(T)`, `PersonChannel(T)`, `RootChannel(T)`, `Conversation(T)`, `SgToApp(T)`, `AppToSg`, `SgChannel(T)`, and the arithmetic-expression type `Exp`; their definitions are in `programs/self.glp` of the public GLP repository, which also declares the entry templates `miniapp/3` and `miniapp_person/5` that a host's guests instantiate (Typed GLP companion paper [50]), and defines `SSchema`, `SContract` and `SDeclared` and the types they are built of, the term form of a contract in the clause syntax of [13], which the contracts of every platform share, the root being the one directory above them all (Typed GLP companion paper, Module System). `Integer`, `Real`, `String`, `Module` and `MutualRef` are primitive types built into the runtime. An `Integer` is a 64-bit two's-complement integer and a `Real` an IEEE 754 binary64 number, and integer arithmetic wraps on overflow, as Dart's does: GLP is implemented in Dart, and its integers are Dart's. A number is written as its digits, with an optional minus directly before them; a minus after an operand is subtraction. An integer and a real are distinct constants, as `Number` is the union of the two: `1` and `1.0` do not match. A functor and its opening parenthesis, and a type or a variable and its `?`, are written with nothing between them, and the reader refuses a program that puts white space or a line break there. `Opaque`, the union of all opaque types, is built into the type checker: a program can only pass a term of it on (Typed GLP companion paper [50]). `Term`, the type of every term, is built into the type checker likewise (Typed GLP companion paper).

Some predefined types name a structure another already denotes. Type identity being structural, the

checker treats such a pair as one type and will accept either where the other is declared; the second name buys no checking. It is declared all the same, for two reasons. It says what a value is for where the structure alone does not — a key and a hash are both strings and are not the same thing to a reader — and it localises a later change of representation to one definition rather than to every declaration that would otherwise have named the structure directly.

Guard predicates. Guards provide read-only access to the runtime state of GLP computation; a defined guard (below) also assigns writers. A guard appears after the clause head, separated by `|`, and must be satisfied for the clause to be selected. The "Ground" column indicates whether success of the guard implies that the argument is ground; such guards permit multiple occurrences of both the writer and the reader in the clause (Definitions 2.18 and 2.19). A guard suspends while no instance of its argument under a readers substitution decides it (Section 2.3).

Guard	Signature	Succeeds when	Ground
`integer`	`procedure integer(Integer?).`	the argument is an integer	yes
`real`	`procedure real(Real?).`	the argument is a real	yes
`number`	`procedure number(Number?).`	the argument is a number	yes
`string`	`procedure string(String?).`	the argument is a string	yes
`constant`	`procedure constant(Constant?).`	the argument is a constant	yes
`module`	`procedure module(Module?).`	the argument is a module	yes
`compound`	`procedure compound(_?).`	the argument is a compound term	no
`list`	`procedure list(_?).`	the argument is the empty list or a list cell	no
`ground`	`procedure ground(_?).`	the argument is ground	yes
`known`	`procedure known(_?).`	the argument is assigned a term	no
`unknown`	`procedure unknown(_?).`	the argument is an unassigned variable	no
`no_readers`	`procedure no_readers(_?).`	no reader occurs in the argument	no
`widowed_writer`	`procedure widowed_writer(_).`	the reader paired to the writer the argument stands for occurs in no goal of the agent and in no link end the agent holds, the goal being reduced not counted	no
`widowed_reader`	`procedure widowed_reader(_?).`	the writer paired to the reader the argument stands for occurs in no goal of the agent and in no link end the agent holds, the goal being reduced not counted	no
`is_mutual_ref`	`procedure is_mutual_ref(MutualRef?).`	the argument is a mutual reference	no
`@<`	`procedure @<(Constant?, Constant?).`	both arguments are ground constants and the first precedes the second in the standard order	yes
`=?=`	`procedure =?=(_?, _?).`	both arguments are ground and equal	yes (both)
`=?\=`	`procedure =?\=(_?, _?).`	no readers substitution makes the arguments ground and equal	no
`otherwise`	(no arguments)	every previous clause of the procedure fails	—

The standard order of constants: a number precedes a string; numbers compare by value, and strings by the codes of their characters, lexicographically; the empty list orders as the string `[]`.

A variable sent to another agent occurs in the link end the agent keeps for it until that end is retired, which the companion implementation paper specifies [48]: an agent that crashed retires every link end at its restart, and its peers retire their ends of the links with it when they learn of the crash. So a writer whose reader was sent away is not widowed while the link stands; a reader whose writer is at a peer that crashed, and a writer whose reader is there, are widowed once the agent learns of the crash, and a

program waiting on either can take a clause guarded by `widowed_reader` or `widowed_writer` in place of the one that waits.

`is_mutual_ref` is admitted only to a system module, as are the body kernels that create, append through and close a mutual reference, the handle through which `mwm/2` appends to the tail of its output stream in constant time: no other module can obtain one, so no other module can put the guard to use.

Arithmetic comparison guards. Arithmetic comparison guards evaluate their arguments as arithmetic expressions of type `Exp` and compare the results; an argument with no value — a zero divisor, a non-integer under `//` or `mod`, an argument outside a function's domain — fails the guard. Success implies both arguments are ground.

Guard	Signature	Ground
`<`	`procedure <(Exp?, Exp?).`	yes
`>`	`procedure >(Exp?, Exp?).`	yes
`=<`	`procedure =<(Exp?, Exp?).`	yes
`>=`	`procedure >=(Exp?, Exp?).`	yes
`=:=`	`procedure =:=(Exp?, Exp?).`	yes
`=\=`	`procedure =\=(Exp?, Exp?).`	yes

Time guards. Time guards provide access to the system clock for scheduling and synchronization:

Guard	Signature
`wait`	`procedure wait(Number?).`
`wait_until`	`procedure wait_until(Number?).`
`when_idle`	(no arguments)

`wait(D)` suspends for `D` milliseconds then succeeds. `wait_until(T)` suspends until the current time is at or past timestamp `T` (Unix milliseconds), then succeeds. `when_idle` suspends while the machine has a Reduce or a Communicate to make, and succeeds when it has none.

Monotonicity and implications. `ground/1`, `no_readers/1`, `known/1`, `widowed_writer/1` and `widowed_reader/1` are monotonic. `ground(X)` implies both `no_readers(X)` and `known(X)`, which do not imply each other: `no_readers(X)` succeeds but `known(X)` fails for an unassigned writer; `no_readers(f(X?))` suspends but `known(f(X?))` succeeds.

Defined guard predicates. To support abstract data types and cleaner code organization, GLP provides for user-defined guards via unit clauses. A unit clause `p(T1,...,Tn).` defines a guard predicate; the call `p(S1,...,Sn)` in guard position is unfolded to the term matching of `T1` with `S1`, ..., `Tn` with `Sn`, the unit clause renamed apart. The guard succeeds, suspends or fails as the matching does, and its writer mgu assigns the writers of the clause among the `Si` and the writers in the values of the readers among them (Definition 2.20): `new_channel(PQCh, QPCh)` assigns the clause's writers `PQCh` and `QPCh` the two ends of a fresh channel, and `send(M?, Ch?, Ch1)` assigns the writer of the output stream in the value of `Ch?` a cell carrying `M?` and `Ch1` the advanced channel. For example, unification is defined by the unit clause `X? = X.`, so the guard `A = B` unfolds to matching both `A` and `B` against the same variable `X`.

The following defined guard predicates are shipped with the runtime:

Predicate	Unit clause
`=`	`X? = X.`
`new_channel`	`new_channel(ch(Xs?, Ys), ch(Ys?, Xs)).`
`send`	`send(X, ch(In, [X?|Out?]), ch(In?, Out)).`
`receive`	`receive(X?, ch([X|In], Out?), ch(In?, Out)).`
`close`	`close(ch([], [])).`
`dl_append`	`dl_append(A\B?, B\C?, A?\C).`
`dl_to_list`	`dl_to_list(L\[], L?).`

Naming and admission of body kernels. A body kernel is named by a quoted constant whose name begins with an underscore, as `'_add'` is. Quoting is necessary: an unquoted name beginning with an underscore is an anonymous variable (Remark 2.22). A module declaring `-mode(system)` may define and call procedures whose names are so written; a module that does not may neither define nor call one. The restriction is on names in call position only; a constant beginning with an underscore is unrestricted as data, as a message tag or a member of a type union. The system modules are the root `programs/self.glp` and the modules under `programs/system/`.

Body kernel predicates. *Body kernel predicates* are runtime-implemented primitives that execute in the clause body with two-valued semantics: they either succeed or abort. Body kernels are not directly accessible to user programs. They are invoked only by the kernel and seam predicates below—predicates defined by GLP clauses that are shipped with the runtime and have privileged access to the body kernel registry. A body kernel whose precondition fails — a zero divisor, an argument out of its domain — aborts.

The table below is the interface between GLP and its runtime, and is documentation for the writers of system modules rather than a facility of ordinary programs: no application module may call any of it. Every kernel the runtime registers appears here, whether or not an ordinary program can reach what it does.

The following table lists all body kernels:

Category	Kernel	Operation
Arithmetic	`'_add'(Number?, Number?, Number)`	Addition; a real where an operand is real
	`'_sub'(Number?, Number?, Number)`	Subtraction; a real where an operand is real
	`'_mul'(Number?, Number?, Number)`	Multiplication; a real where an operand is real
	`'_div'(Number?, Number?, Real)`	Division
	`'_idiv'(Number?, Number?, Integer)`	Integer division; aborts on a non-integer operand
	`'_mod'(Number?, Number?, Integer)`	Modulo; aborts on a non-integer operand
	`'_abs'(Number?, Number)`	Absolute value; a real of a real
Random	`'_random'(Integer?, Integer?, Integer, Integer)`	Seeded random integer
	`'_split_random_seed'(Integer?, Integer, Integer)`	Two independent seeds from one
Math	`'_sqrt'(Number?, Real)`	Square root
	`'_sin'(Number?, Real)`	Sine
	`'_cos'(Number?, Real)`	Cosine
	`'_tan'(Number?, Real)`	Tangent
	`'_asin'(Number?, Real)`	Arc sine
	`'_acos'(Number?, Real)`	Arc cosine
	`'_atan'(Number?, Real)`	Arc tangent
	`'_exp'(Number?, Real)`	Exponential (e^x)
	`'_ln'(Number?, Real)`	Natural logarithm
	`'_log10'(Number?, Real)`	Base-10 logarithm
	`'_pow'(Number?, Number?, Number)`	Power (x^y); a real where an operand is real or the exponent negative
Conversion	`'_integer'(Number?, Integer)`	Convert to integer
	`'_real'(Number?, Real)`	Convert to real
	`'_round'(Number?, Integer)`	Round to nearest integer
	`'_floor'(Number?, Integer)`	Floor
	`'_ceil'(Number?, Integer)`	Ceiling
Structure	`'_list_to_tuple'(Stream(_)?, _)`	List to compound term
	`'_tuple_to_list'(_?, Stream(_))`	Compound term to list
Text	`'_string_concat'(List(String)?, String)`	Concatenation of strings
	`'_term_to_string'(_?, String)`	The text the reader reads as a term
Time	`'_now'(Integer)`	Current Unix timestamp (ms)
Simulation	`'_start_clock'(Integer?)`	Start the simulated clock
	`'_rated'(Number?, Token)`	A token pending for an exponential delay
	`'_step_clock'(Number?, ClockStep)`	Release the earliest pending token, or stop
I/O	`procedure(X) '_output'(X?).`	Send term to output callback
MWM	`procedure(X) '_allocate_mutual_reference'(MutualRef, Stream(X)).`	Create mutual reference for O(1) append
	`procedure(X) '_stream_append'(MutualRef?, X?, MutualRef).`	Append value via mutual reference
	`procedure '_close_mutual_reference'(MutualRef?)`	Close stream (bind

Kernel and seam predicates. A predicate defined by GLP clauses whose body may invoke a body kernel is of one of two kinds, and the difference is what lies on the other side of it. A *kernel predicate* reaches the runtime that implements GLP: arithmetic, term structure, the clock, the module system. A *seam predicate* reaches something the language cannot reach at all — the person, the person's keys, the networking layer, the platform. The first kind is GLP realised; the second is GLP joined to what is not GLP, and a seam is where two things are joined.

The distinction is not one of mechanism: both are ordinary GLP clauses, both may name a body kernel in call position, and both are admitted only to a system module. It decides where the specification of a construct's behaviour belongs. A kernel predicate's behaviour is the language's own and is stated here. A seam predicate's is not: what an address is, what a geofence reports, what a signature attests are settled by the paper that specifies the thing on the far side, and this catalogue states the predicate and cites that paper.

Both tables list those in root scope, where every program reaches them; a system module may define others for its own use, and those belong to that module rather than to the language.

A program declares every procedure it defines, with no `_` or `_?` in a declaration or in a type definition; the two stand only in a signature of these tables that takes any term, as `run/2` and `..=` do.

Kernel predicate	Description
`procedure =..(_, Stream(_)?).`	Term composition
`procedure ..=(Stream(_), _?).`	Term decomposition
`procedure string_concat(List(String)?, String).`	Concatenation of strings
`procedure term_to_string(_?, String).`	The text the reader reads as a term
`procedure now(Integer).`	Current Unix timestamp (milliseconds)
`procedure :=(Number, Exp?).`	Arithmetic evaluation and assignment
`procedure random(Integer?, Integer?, Integer, Integer).`	Seeded random integer
`procedure split_random_seed(Integer?, Integer, Integer).`	Two independent seeds from one
`procedure start_clock(Integer?).`	Start the simulated clock
`procedure rated(Number?, Token).`	A token pending for an exponential delay
`procedure step_clock(Number?, ClockStep).`	Release the earliest pending token, or stop
`procedure self_module(Module).`	The caller's own module, as a value
`procedure run(_?, Module?).`	Activate a module value, post a goal to it
`procedure run(_?, Constant?, Module?).`	As `run/2`, under a required type identity
`procedure find_type(_?, Constant).`	The type identity of a declaration
`procedure find_type(_?, List(Constant)?, Constant).`	The type identity of an opaque instance
`procedure opaque_type(_?, Module?, List(Constant)).`	The types a module records for its entry
`procedure(X) mwm(MwmInput(X)?, Stream(X)).`	Multiway merge of many streams into one
`procedure(X) allocate_mutual_reference(MutualRef, Stream(X)).`	Allocate a mutual reference on a stream's tail
`procedure(X) stream_append(X?, MutualRef?, MutualRef).`	Append a value through a mutual reference
`procedure close_mutual_reference(MutualRef?).`	Close the stream a mutual reference holds

Seam predicate	Description
`procedure(X) send_to_user(Stream(X)?).`	Emit each term of a stream to the person
`procedure(X) send_to_net(Stream(NetMsg(X))?).`	Send each message of a stream to the agent it names
`procedure sign(_?, Key?, SignedTerm).`	Sign a term under a named key
`procedure authorise_link(GlobalName?, Answer?).`	Release or drop a reported held link
`procedure peer_address(Key?, PeerAddress).`	Address at which the layer observes a peer
`procedure peer_app(Key?, App).`	Developer, name and version of the app a peer attested
`procedure self_app(App, App).`	The app whose program runs, and the app installed
`procedure punch_udp(String?).`	Open a path to an address
`procedure declare(Declaration?).`	A peer, an address, a rendezvous server, a trust level, a place
`procedure remove(Removal?).`	The end of a declaration
`procedure self_key(Key).`	The agent's public key
`procedure signature(_?, Signature).`	Signer, signing module and what was signed, or `unsigned`
`procedure decompose_module(Module?, Key, Hash, Hash).`	Compiler's key, source and compiled identity
`procedure compile(String?, Compiled).`	Compile the program a path names
`procedure load_file(String?, Content).`	Read a file in the calling module's directory

Term composition and decomposition (`=..`, `..=`). Composition and decomposition are two predicates, not one bidirectional predicate. A declaration fixes one mode per argument, so a single name cannot carry both directions: `=..` takes a list and produces a term, and `..=` takes a term and produces a list, each with the opposite mode on both arguments. `T =.. [foo, a, b]?` constructs `T = foo(a, b)`; `L ..= foo(a, b)?` produces `L = [foo, a, b]`.

Both are needed, and a program that walks an arbitrary term needs both together: `..=` to take a compound apart and recurse on its arguments, `=..` to rebuild it from the results. Decomposition alone descends without returning, and composition alone cannot get started.

Clock access (`now/1`). Binds the argument to the current Unix timestamp in milliseconds since the epoch.

Arithmetic evaluation and assignment (`:=`). Arithmetic assignment is defined recursively: each clause pattern-matches on an arithmetic operator, guards verify that operands are of the required type, and the body invokes the corresponding body kernel. An `otherwise` clause handles the case where operands are themselves expressions, evaluating them recursively before retrying.

Seeded random numbers (`random/4`). `random(Seed, N, Value, NextSeed)` binds `Value` to a pseudo-random integer with $1 \leq$ `Value` $\leq$ `N`, and `NextSeed` to the seed of the next draw, given a ground seed `Seed` and a ground bound `N` ≥ 1. It is a body goal, not a guard; the clause is guarded by `integer(Seed?), integer(N?), N? >= 1` and invokes the `'_random'` body kernel. `NextSeed` is determined by `Seed` alone; `Value` by `Seed` and `N`. The seed stream is therefore independent of the bound requested, and a computation that threads `NextSeed` from each draw into the next reproduces exactly from its initial seed. The generator is SplitMix64 [17], with its root increment and its mixing function, in 64-bit arithmetic modulo 2^{64}: a seed is a 64-bit integer, and every 64-bit integer is a seed; `NextSeed` is `Seed` plus the increment `0x9E3779B97F4A7C15`; and `Value` is one plus the mix of `NextSeed`, taken as an unsigned 64-bit integer, modulo `N`, where the mix of z is $z \leftarrow (z \oplus (z \gg 33)) \cdot$ `0xFF51AFD7ED558CCD`, then $z \leftarrow (z \oplus (z \gg 33)) \cdot$ `0xC4CEB9FE1A85EC53`, then $z \oplus (z \gg 33)$, with $\gg$ the unsigned shift.

Splitting a seed (`split_random_seed/3`). `split_random_seed(Seed, S1, S2)` assigns `S1` and `S2` two seeds determined by `Seed`, such that the sequences of draws from `Seed`, from `S1` and from `S2` are independent streams: `S1` is the mix of `Seed` plus the increment, and `S2` the mix of `Seed` plus twice the increment, the increment and the mix those of `random/4`. It is a body goal, not a guard; the clause is guarded by `integer(Seed?)` and invokes the `'_split_random_seed'` body kernel. A computation that draws from several seeds made by splitting one seed reproduces exactly from that seed.

Simulated time (`start_clock/1`, `rated/2`, `step_clock/2`). A machine has a *simulated clock*, a real number of seconds, and a queue of *pending tokens*, each with an activation time; `now/1` and the time guards read the system clock. The types are `Token ::= go(Number) ; stop` and `ClockStep ::= released(Number) ; stopped(Number)`. `start_clock(Seed)` binds every pending token to `stop`, sets the clock to 0, and seeds the clock's generator with `Seed`; the clause is guarded by `integer(Seed?)`. `rated(Rate, Tok)` makes the token `Tok` pending, with activation time the clock plus D, where D is drawn from the exponential distribution with rate `Rate` per second; the clause is guarded by `number(Rate?), Rate? > 0`. `step_clock(Horizon, Step)` takes one step of the clock, and the clause is guarded by `number(Horizon?)`. Where a token is pending whose activation time T is the least and at most `Horizon`, the step sets the clock to T, binds that token to `go(`T`)` and assigns `Step` the term `released(`T`)`; two tokens of one activation time are released in the order in which they were made pending. Otherwise the step binds every pending token to `stop` and assigns `Step` the term `stopped(`C`)`, C being the clock. Each of the three is a body goal that invokes the body kernel of its name, and `rated` and `step_clock` abort before the first `start_clock`.

The clock's generator is that of `random/4`, its state a seed. A draw replaces the state by the state plus the increment, and $D = -\ln(U)/\texttt{Rate}$, where U is one plus the mix of the new state, taken as an unsigned 64-bit integer and shifted right by 11 bits, over 2^{53}, so that $0 < U \leq 1$. A computation that makes its tokens pending in the same order reproduces exactly from the seed.

A program takes a step in the body of a clause guarded by `when_idle`: the guard tests that the machine has no Reduce or Communicate to make, and the step is taken after it, a guard changing nothing. So every goal has been reduced as far as it can be at the present simulated time before the clock moves, and a goal waiting on a token is reduced once its time has come. The clock is of one machine, as `when_idle` is. The three predicates implement the rated goal and the Release transition of Stochastic GLP [49]: the queue, the draws and the clock are the engine's, as they were the emulator's in the implementation of the stochastic pi-calculus in Flat Concurrent Prolog [31], by Gillespie's method [11].

Output to the person (`send_to_user/1`). `send_to_user(Stream)` emits each element of `Stream`, a stream of any one type, to the person operating the agent, in order, and ends when the stream ends. Its clause is guarded by `ground` on the element and invokes the `'_output'` body kernel. A module reaches the person through this predicate; naming `'_output'` in call position is admitted only to a system module. Where the person is a terminal, each element is printed on a line of its own, a string as its characters.

Text (`string_concat/2`, `term_to_string/2`). `string_concat(Ss, S)` assigns `S` the string of the characters of the strings of the list `Ss`, one after another; the empty list contributes the characters `[]`. Its clause is guarded by `ground(Ss?)` and invokes the `'_string_concat'` body kernel, which aborts where an element is not a string. `term_to_string(T, S)` assigns `S` a text that the reader reads as the ground term `T`: a string the reader would read unquoted as anything else in single quotes, a number as the reader reads it back, a structure as its functor so written, an opening parenthesis, its arguments and a closing parenthesis, and a list as `[a, b]` or `[a, b | T]`, arguments and elements separated by a comma and a space, so that a term has one text at every agent. Its clause is guarded by `ground(T?)` and invokes the `'_term_to_string'` body kernel. A program that prints GLP source, as a compiler does, builds each line as a string by these two, a constant of the source by `term_to_string/2` and a variable or a type `T?` by `string_concat/2`, and prints it by `send_to_user/1`.

Output to the network (`send_to_net/1`). `send_to_net(Stream)` sends each element of `Stream`, a term `msg(Q, T)` of type `NetMsg(X) ::= msg(Key, X)` with `Q` the key of an agent, to agent `Q`, in the order of the stream, which the layer delivers fairly and in any order [2], and ends when the stream ends: it

is how a program writes the network output stream of Definition 3.6, whose Cold-call carries `msg(Q, T)` to the network input stream of `Q`. Its clause is guarded by `ground` on `Q` and invokes the `'_send'` body kernel on `msg(Q?, T?)`, the link `'_w'(Q?, 0, 0)` and `Q?`. A module reaches the network through this predicate; naming `'_send'` in call position is admitted only to a system module.

Module as a value (`self_module/1`, `decompose_module/4`). `self_module(Module)` binds its argument to the caller's own module, a value of the primitive type `Module`. `decompose_module(Module, Key, Source, Co` takes a module value apart, assigning the public key of the compiler that produced it, the identity of the source it was compiled from, and the identity of the compiled module itself.

The two identities are distinct and neither determines the other. Source identity says what was written; compiled identity says what runs, and two compilers compiling one source produce two of them. The compiler's key is what joins them: it is the compiler that vouches for the correspondence, and a party relying on the pair decides for itself whether that key is one it accepts. A module value is opaque to a program, so the decomposition has to come from the runtime. Neither identity is the type identity of `run/3` and `find_type/2`, which is a property of a declaration and not of a module.

Dynamic activation (`run/2`, `run/3`, `find_type/2`, `find_type/3`, `opaque_type/3`). `run(Goal, Module)` activates the module value `Module` and posts `Goal` to it; the predicate of `Goal` must be exported by `Module`, else an error. The readers and writers of `Goal` connect the caller and the activated module. `run(Goal, Type, Module)` activates `Module` and posts `Goal` to it where `Type` equals the type identity that `Module`'s exports record for `Goal`'s predicate, and errs otherwise. `find_type(P/N, Type)` assigns `Type` the type identity of the declaration of `P/N` in the caller's scope, and `error` where `P/N` is not declared there. Type identity is defined in the Typed GLP companion paper [50], which also gives its construction and proves the compatibility it rests on. `find_type(P/N, Hs, Type)` assigns `Type` the type identity of the opaque instance at `Hs` of the caller's parameterised declaration of `P/N`, `Hs` a list of type identities, one per type parameter of the declaration in declaration order, and errs where `P/N` is not declared there or the length of `Hs` is not the number of its type parameters. `opaque_type(P/N, Module, Hs)` assigns `Hs` the list of the type identities of the types at which `Module`'s exports record its entry `P/N` as the instance of a parameterised declaration, in the order of its parameters, and errs where they record none. A host activates a guest it was not compiled with by `opaque_type(P/N, M, Hs)`, `find_type(P/N, Hs, Type)` and `run(Goal, Type, M)`, holding no declaration of the guest's own types. At a position whose type has a parameter of the template in it, `Goal` may hold a variable whose type has `Opaque` in its place, at which a host holds the channel ends of its guests. The opaque instance, `Opaque`, the invitation discipline under which this is sound, and its proof are in the Typed GLP companion paper.

Compilation and file reading (`compile/2`, `load_file/2`). `compile(Source, Compiled)` compiles the program the path `Source` names, under the root the device nominates, and assigns `Compiled` either `module(M)` for the compiled module `M` or `error(Es)` for the stream of diagnostics that rejected the program; where it succeeds it also writes `M` as an artefact in that program's own directory. `load_file(Name, Content)` reads the file `Name` names and assigns `Content` the module that file holds where it is a certified compiled module, and its text otherwise. The types are `Compiled ::= module(Module) ; error(Stream(String))` and `Content ::= String ; Module`, the caller discriminating with the `string` and `module` guards. What a program is, what compilation does to one, and what the root is are specified in the Typed GLP companion paper [50].

A module value therefore cannot be forged by writing a file: a file whose certificate does not check is text and is not a `Module`, which is what `decompose_module/4` rests on.

`Name` is resolved within the calling module's own directory, and the caller can neither escape it nor choose otherwise. That directory is the module's path from the root, which compilation assigns it, and it is the runtime's contribution for the same reason the module identity in a signature is. A module needing something outside its own area receives it from whoever holds it, as a value or over a stream, rather than by naming it. Neither predicate admits a path where a module value is expected, a path being resolved by the device and carrying none of the identities `decompose_module/4` takes apart; `run/2` and `run/3` take the value. `compile/2` is not yet implemented.

Identity and signature (`self_key/1`, `sign/3`, `signature/2`). The runtime makes the agent's key pair when it spawns the agent, before anything is compiled or sent, from a seed drawn from outside the program, as the companion implementation paper says, and gives it to the networking layer, which holds the private key and signs with it [2]. A `Key` is the 64-character lowercase hexadecimal of the 32-byte Ed25519 public key, so `@<` orders keys as their bytes. `self_key(Key)` assigns `Key` the agent's public key. `sign(Term, Key, Signed)` requires `Term` ground and `Key` a key of that person, and assigns `Signed` the signature under `Key` of the pair of `Term` and the source identity of the module making the call; `Key` is the agent's own, the layer holding no other, and any other key aborts the call. A signed term is a string, so it travels as text, as an invitation link does [2]. `signature(Signed, Signature)` takes a signed term apart and reports what it finds. Where `Signed` is a signed term whose signature verifies, it assigns `Signature` the term `signed(Key, Hash, Term)`, giving the signer's key, the identity of the module under which the signature was made, and the term signed; otherwise it assigns the constant `unsigned`. The type is `Signature ::= signed(Key, Hash, _) ; unsigned`, and the caller discriminates by matching, taking `signed(Key, _, _)` where it needs to know who signed and not what. A predicate that did not hold on a forgery could not refuse one and carry on, which is what an agent presented with an invitation, an undertaking or a creator's certification does.

The module identity is the runtime's contribution and cannot be the program's, because a program can build any term it likes. Were the signature over `Term` alone, any module the person runs could obtain that person's signature on any statement whatever, and a signature would attest only that the person ran something. Scoped, it says that the person ran this module and that this module asked for this term, which is what a relying party needs.

A signature is what carries a claim to a party that has neither an authenticated channel to its author nor an attestation of the runtime that produced it. Within a conversation none is needed: authorship of every stream extension is already established to its reader by the channel and by the attested runtime at the writer's end. It is needed outside one — a rendezvous agent belongs to no social graph and is itself unattested, and admits a connecting agent on a signed capability and on nothing else.

Held links (`authorise_link/2`). `authorise_link(L, A)` answers a link that the runtime holds and reports because its other end is at an agent with which the name was not exchanged. `L` is the global name of the reported link, of type `GlobalName`, and `A` is of type `Answer`, either `authorise` or `refuse`: on `authorise` the runtime releases the held message or assignment, on `refuse` it drops the link. The type is `GlobalName ::= '_r'(Key, Integer, Integer) ; '_w'(Key, Integer, Integer)`, naming the reader or the writer end of a link at the agent whose key it carries, by that agent's epoch and the index it allocated in it [48], and it is the type of the link `'_send'` names likewise. The clause is guarded by `known(A?)` and invokes the `'_authorise_link'` body kernel. A link end is held when the agent at its other end is not the one with which the name was exchanged; while it is held the runtime places nothing on its channel and applies nothing arriving on it. The decision is the program's rather than the runtime's because the reason to release or drop such a link is not visible to the runtime, which therefore reports and waits. The mechanism that holds a link and reports it is described in the companion

implementation paper [48].

Networking seam (`peer_address/2`, `peer_app/2`, `self_app/2`, `punch_udp/1`, `declare/1`, `remove/1`). `peer_address(P, A)` assigns `A` `address(S)`, `S` the address, `ip:port`, at which the networking layer observes peer `P` on its live session over the WAN, or else over the LAN, or `none` where it has no live session with `P` [2]; `PeerAddress ::= address(String) ; none`. `peer_app(P, A)` assigns `A` `app(D, N, V)`, the developer `D`, the name `N` and the version number `V` of the app attested in the networking layer's session with peer `P`, the session reported last where `P` is connected over several underlays, `unattested` where that session is unattested, or `none` where the layer has no session with `P`; `App ::= app(String, String, Integer) ; unattested ; none`. `self_app(R, I)` assigns `R` `app(D, N, V)` for the app whose program the agent runs, the app the networking layer attests to the agent's peers, and `I` the same for the app installed on the device; the two differ after an update the runtime refused [48], and where the runtime is given no app, as in a simulation or on a headless server, each is `none`. `punch_udp(A)` opens a path to address `A` and assigns nothing. `declare(D)` makes the declaration `D` to the networking layer or the platform, and `remove(R)` ends one: `Declaration ::= peer(Key) ; address(Key, String) ; rendezvous(Key, String) ; trust(ProximityU` and `Removal ::= peer(Key) ; rendezvous(Key) ; place(Constant) ; nearby(ProximityUnderlay) ; re` with `ProximityUnderlay ::= pan ; lan`, `Network ::= attached(String) ; none`, `TrustLevel ::= open ; closed`, `PlaceEvent ::= entered ; exited ; observable ; unobservable` `Presence ::= seen(Key) ; gone(Key)` and `Reach ::= connected(Key) ; lost(Key)`. Each clause is guarded by `ground` on what it consumes, and the clauses themselves are in `programs/self.glp`.

`peer(P)` gives the networking layer the public key `P` of a peer the program knows, and `remove(peer(P))` takes `P`, and any address recorded for it, off the layer's known peers and ends its sessions on every underlay; `address(P, A)` gives the layer the address `A` at which peer `P` is reached, making `P` a known peer, and an address the layer observes later replaces it. The layer contacts over the WAN only a known peer and a declared rendezvous server [2]. Who learns an agent's address is the social graph's decision, and the layer holds no social graph, so the program tells the layer what it knows. `rendezvous(P, A)` gives the layer a rendezvous server, the agent of key `P` at address `A`, or replaces its address, keeping a session standing with it, and `remove(rendezvous(P))` takes the server off and ends the layer's session with it, a key not declared doing nothing; the program knows a rendezvous server by the key it declared, which tells it from an unattested phone, the layer reporting no app for either.

`place(P, R, E)` declares a place `P` of radius `R` metres, centred on the declaring agent's location at the declaration, which centre the runtime keeps and declares the place about again at a restart [48], and assigns `E` a stream of that agent's own `entered` and `exited` events for the place, interleaved with `unobservable` where the layer has stopped reporting crossings and `observable` where it resumes; between the two no crossing is seen and none is delivered late, and a place declared while reporting is stopped begins with `unobservable`. A place the layer cannot register — a radius that is not a positive number, a place the platform refuses, any place on a headless server, which has no location service — stands declared, its stream beginning with `unobservable` [2, 48]. A further declaration of the same place supersedes the earlier one and closes its stream. `remove(place(P))` ends the declaration of `P`, and does nothing where `P` is not declared.

`nearby(U, S)` assigns `S` a stream of the agents nearby over the proximity underlay `U`, beginning with `seen` for every agent nearby when it is declared: `seen(K)` when the layer discovers on `U` an agent of key `K` that it recognises or, under open trust, contacts, and `gone(K)` when its presence is no longer observed on `U` and the layer holds no session with it there [2]. A further declaration on the same underlay

supersedes the earlier one and closes its stream, and `remove(nearby(U))` ends the declaration and closes its stream. The agents seen over the PAN are the physically near ones, so a program learns which of its friends are near from this stream [2].

`reachable(S)` assigns `S` a stream of the agents the networking layer connects to over the WAN: `connected(K)` when the layer's session with the agent of key `K` over the WAN is established, authenticated and attested, a declared rendezvous server being reported too, unattested [2], beginning with every agent it has one with when it is declared, and `lost(K)` when that session ends. A further declaration supersedes the earlier one and closes its stream, and `remove(reachable)` ends the declaration and closes its stream.

`public_address(S)` assigns `S` a stream of the agent's own public address, beginning with `address(A)`, `A` the `ip:port` the layer has learnt from a rendezvous server, or `none` where it knows none, when it is declared, and then the address at each change, `none` where it is lost. An agent cannot observe its own public address, which changes as it roams; a program sends it to its friends over the channels that remain, so that each declares it to its layer and reconnects [2]. `network(S)` assigns `S` a stream of the local network the agent is attached to, beginning with `attached(N)`, `N` an opaque fingerprint of that network, or `none` where it is attached to none or the platform makes the fingerprint unreadable, when it is declared, and then the network at each change; the fingerprint is best-effort, two networks possibly giving one, so a program compares it only with its own earlier ones, as one that opens LAN trust on a network its person declared trusted does [2]. For each, a further declaration supersedes the earlier one and closes its stream, and the removal ends the declaration and closes its stream. The address and the network change independently of each other.

An agent whose layer does not run an underlay declares nothing on it: a trust level for it has no effect and its stream carries nothing [2].

`peer_address/2` and `punch_udp/1` are on this seam because a GLP program cannot reach the platform by itself: `peer_address/2` is address discovery, an agent behind a NAT learning from a peer the public address that peer observes for it, and `punch_udp/1` opens a path to a peer so observed, the two together standing in for external address-discovery infrastructure. A place declaration registers a geofence with the platform's location service. Crossings reach the program as messages on a stream rather than through a guard, and must: a guard is a test on the resolvent alone, so one that consulted the device's location would let the same local state reduce differently at different times. A declaration carries a name and a radius and returns no coordinates, so a program learns of its own device's location only that it entered or left a place it named itself. The runtime behind the kernels they invoke is described in the companion implementation paper [48]; the layer beneath that is specified in the networking companion paper.

Proximity trust. A *proximity underlay* carries a *cold-call trust level*, which decides whether the agent contacts, and answers, a device it does not already know [2]. `trust(U, L)` declares that level for one underlay: `U` is `pan` (Bluetooth Low Energy) or `lan` (the local network) and `L` is `open` or `closed`. Both underlays stand at `closed` until declared, a later declaration of an underlay replaces the level then standing, and the two are declared independently, so an agent may answer strangers over one and ignore them over the other.

The level is the program's to set and cannot be the layer's, for a reason that decides where the predicate belongs: an agent raises the level on entering a place it trusts and lowers it on leaving, and lowers it again when that place's crossings become unobservable, doubt resolving to closed — and the layer cannot do any of that, because it does not know which place is trusted. The program declared the place.

Multiway merge (`mwm/2`). `mwm(In, Out)` merges the streams named by `In` into the single stream `Out`, preserving the order of each input stream and interleaving them fairly. `In` is a stream of `merge(S)` elements, each contributing the stream `S` to the merge, following the merger of FCP [39]. Since `In` is itself a stream, a `merge(S)` element arriving later grafts a further stream into a running merge: the set of merged streams need not be known when the merge begins, and a merge is closed by closing `In`.

`mwm/2` is the construct to use whenever many streams are merged into one, and is preferred to a tree of `merge/3` goals for that purpose. The difference is asymptotic, not stylistic. Merging n streams with binary `merge/3` passes each message through $\log n$ merge processes on a balanced tree, and through as many as n on the linear tree a program most easily writes; `mwm/2` appends each message once, in constant time, through a handle on the tail of `Out`. A program that builds its own merge tree pays a factor no implementation can recover, since the cost is in the process structure the program chose. None of this makes a hand-built merge tree wrong: it is sound, and knowing how to build one is worth having. The preference is about which construct a program should reach for when many streams must arrive at one receiver, not about which technique is worth understanding.

`merge/3` remains the right construct where two streams are merged, or where the merge tree itself carries meaning — a merge goal that is also the point at which a process terminates. `mwm/2` is for many-to-one merging, which is the common case in a program routing many senders to one receiver.

The complete definitions of `:=`, `now`, `=..`, `..=`, `string_concat`, `term_to_string`, `random`, `split_random_seed`, `start_clock`, `rated`, `step_clock`, `send_to_user`, `send_to_net`, `self_module`, `decompose_module`, `run`, `self_key`, `sign`, `signature`, `authorise_link` and `mwm` are in `programs/self.glp` of the public GLP repository.

E Deferred Proofs

This appendix contains deferred proofs from Sections 2, 3, and 5.

E.1 cGLP Proofs

Proposition E.1 (cGLP Computation is Deduction)**.** *Let $(G_0 \ :\!-\ G_n)\sigma_n^\star$ be the outcome of a proper cGLP run $\rho : (G_0, \sigma_0) \to \cdots \to (G_n, \sigma_n)$. Then $(G_0 \ :\!-\ G_n)\sigma_n^\star/?$ is a logical consequence of $P/?$.*

Proof. The $/?$ operator replaces every reader $X?$ by its paired writer X, transforming GLP terms into LP terms. We show that the cGLP run ρ corresponds to an LP run $\rho/?$ of $LP(P/?)$.

Consider a cGLP transition $(G, \sigma) \to (G', \sigma')$:

- *Reduce transition*: Goal A reduces with clause C via writer mgu $\hat{\sigma}$, and $(\hat{\sigma}?)^\star$ is applied to the body. Applying $/?$, the clause $C/?$ is an LP clause, and $A/?$ unifies with the head $H/?$ via the mgu $\hat{\sigma}/? = (\hat{\sigma}?)^\star/?$ (since writers map to writers), which LP applies to the body. This is a valid LP reduction.
- *Communicate transition*: A reader $X? \in G$ is replaced by the value T assigned to its paired writer. Under $/?$, both $X?$ and X map to X, so this transition becomes the identity—the variable X is already assigned T in the LP view.

Thus each cGLP transition corresponds to zero or one LP transitions, and the cGLP run ρ projects to an LP run $\rho/?$ of $LP(P/?)$. By standard LP soundness, the outcome of $\rho/?$ is a logical consequence of $P/?$. □

Proposition E.2 (SO Preservation)**.** *If the initial goal G_0 satisfies SO, then every goal in a proper cGLP run satisfies SO.*

Proof. By induction on the length of the run. The base case is immediate: G_0 satisfies SO by assumption.

For the inductive step, assume G satisfies SO and consider a transition $(G,\sigma) \to (G',\sigma')$:

- *Reduce transition*: Goal $A \in G$ reduces with clause $C = (H \text{ :- } B)$ via writer mgu $\hat{\sigma}$, yielding $G' = (G \setminus \{A\}) \cup B(\hat{\sigma}?)^\star$. Since C satisfies SRSW, it satisfies SO. Since C is renamed apart from G, the variables in B are fresh. $\hat{\sigma}$ binds the writers of H to pairwise disjoint subterms of A and the writers of A to pairwise disjoint subterms of H, a writer of H occurs nowhere in B and its reader at most once in C (by SO of C), and a reader of a writer of A occurs at most once in G (by SO of G), so $B(\hat{\sigma}?)^\star$ satisfies SO, and no variable of A occurs in $G \setminus \{A\}$ (by SO of G). Since $G \setminus \{A\}$ satisfies SO, B satisfies SO, and no variable of B occurs in G (by renaming apart), G' satisfies SO.
- *Communicate transition*: $G' = G\{X? := T\}$ with $\{X? := T\} \in \sigma$. Since G satisfies SO, $X?$ occurs at most once in G. Replacing that occurrence by T, which satisfies SO (Definition 2.6), preserves SO provided no variable of T occurs elsewhere in G. A variable occurs in at most one term σ assigns: a clause variable occurs once in its clause, and matching binds the writers of a head to pairwise disjoint subterms of the goal. A variable of an assigned term enters a goal only by the Communicate of that assignment, since a Reduce brings into the goal the variables of a clause renamed apart, which by the proper run condition (Definition 2.12) are new to the configuration, and variables of the reduced goal, which it removes. So no variable of T occurs in G, and G' satisfies SO.

□

Proposition E.3 (Monotonicity)**.** *In any proper cGLP run, if unit goal A can reduce with clause C at step i, then either an instance of A has been reduced by step $j > i$, or an instance of A can still reduce with C at step j.*

Proof. Suppose goal A can reduce with clause C at step i, meaning the writer mgu of A and the head H of (a renaming of) C succeeds. Consider what can change between steps i and $j > i$:

- *Reduce transitions on other goals*: These do not affect A directly. By SO, no other goal shares a writer with A, so no other reduction can assign a writer in A.
- *Communicate transitions*: These instantiate readers, not writers. A Communicate transition $X? := T$ may instantiate a reader $X? \in A$, yielding $A' = A\{X? := T\}$. We show A' can still reduce with C:

  The original writer mgu succeeded, meaning at position p where $X?$ occurred in A, H had a writer Y at position p, yielding assignment $Y := X?$; a reader or a non-variable term of H at position p would have caused failure or suspension, respectively, contradicting the success of the writer mgu.

  After the Communicate transition, A' has T at position p. The clause C (renamed apart from A') has a fresh writer Y' at position p. The writer mgu now yields $Y' := T$, which succeeds.
- *Reduce transition on A*: If A itself is reduced at some step k with $i < k \leq j$, then an instance of A has been reduced, satisfying the proposition.

Thus, if A has not been reduced by step j, the (possibly instantiated) goal A' at step j can still reduce with a fresh renaming of C. □

Corollary E.4 (SO Preservation with Guards)**.** *Proposition 2.14 holds for programs of guarded clauses satisfying SRSW.*

Proof. Let a Reduce transition reduce A with the renamed guarded clause $H \text{ :- } G \mid B$ via writer mgu $\hat{\sigma}$. For each ground-guarded writer X, the success of $G\hat{\sigma}?$ implies that $X?\hat{\sigma}?$ is ground (Definition 2.18). So $\bar{\theta} = \theta \cup \theta?$, with θ the restriction of $(\hat{\sigma}?)^\star$ to the ground-guarded writers, is a ground instantiation of the clause, and $(H \text{ :- } B)\bar{\theta}$ satisfies SO (Definition 2.19). A and $H\bar{\theta}$ have the writer mgu $\hat{\sigma}'$ obtained from

$\hat{\sigma}$ by removing the bindings of the ground-guarded writers and applying $\bar{\theta}$ to its terms, and $B(\hat{\sigma}?)^\star = (B\bar{\theta})(\hat{\sigma}'?)^\star$. So the transition is the Reduce of A with the clause $(H \text{ :- } B)\bar{\theta}$, which satisfies SO, and a guard that is not defined adds no variable to the configuration; the proof of Proposition 2.14 applies. A defined guard's $\hat{\tau}$ assigns writers of the clause, and writers in the values of its readers, terms of a unit clause renamed apart, which satisfies SO and whose variables are fresh; so $B(\hat{\sigma} \cup \hat{\tau})?^\star$ satisfies SO as $B(\hat{\sigma}?)^\star$ does, and the assignments of $\hat{\tau}$ to the writers of A are communicated as those of $\hat{\sigma}$ are, each to the one occurrence of its reader. □

E.2 maGLP Proofs

Proposition E.5 (maGLP SO Preservation)**.** *If the initial goals of all agents satisfy SO, then every goal in every agent's resolvent throughout a proper run satisfies SO.*

Proof. Reduce p preserves SO at agent p by the same argument as cGLP SO Preservation (Proposition 2.14), restricted to agent p's resolvent. Communicate p to q removes $\{X? := T\}$ from σ_p (no change to G_p) and applies $\{X? := T\}$ to G_q. The substitution preserves SO in G_q provided no variable of T occurs in G_q; this holds as in the cGLP case: a variable occurs in at most one assigned term, and enters a goal only by the Communicate of that assignment. For Cold-call p to q, the message $\texttt{msg}(q,X)$ is transferred from p's network output stream to q's network input stream. The writers in X are no longer reachable from p's resolvent after advancing (the resolvent has iterated past the consumed stream element), so they occur only in q's resolvent. Input p adds $\{X? := T\}$ to σ_p and changes no goal; the variables of T are fresh, so they occur in no goal until the Communicate of that assignment, which is the cGLP case. SO is preserved. □

Proposition E.6. *Every finite proper maGLP run r over P and M is simulated by a proper cGLP run of L_r whose outcome is a logical consequence of $M_r'/?$.*

Proof. Let $r = c^0 \to c^1 \to \cdots \to c^n$ be a finite proper maGLP run over P and M, with $c_p^k = (G_p^k, \sigma_p^k)$ for $p \in P$. We construct a proper cGLP run $\hat{r}$ of L_r such that, after simulating the k-th maGLP step, the goal of $\hat{r}$ contains $\bigcup_{p\in P} G_p^k$ and its substitution contains $\bigcup_{p\in P} \sigma_p^k$.

The initial goal of L_r is $\bigcup_{p\in P} G_p^0$ together with the `network` goal and the $\texttt{person}_p$ goals. It satisfies SO: within each agent's initial goal every variable occurs once, the agents' initial goals are variable-disjoint, the variables of each agent's being its own (Definition 3.5), and each agent's network-channel variables are paired one-for-one with those of the `network` goal, and its person-channel variables with those of its $\texttt{person}_p$ goal. Hence by SO Preservation (Proposition 2.14) every goal of $\hat{r}$ satisfies SO. We simulate each maGLP transaction.

Reduce p. The step is a cGLP Reduce of some $A \in G_p^k$ with a clause $C \in M \subseteq M_r'$. Since G_p^k is part of the goal of $\hat{r}$ and $C \in M_r'$, the identical reduction is a cGLP Reduce step of L_r, updating the goal and the substitution exactly as in maGLP and leaving the other agents' goals and the `network` and `person` goals untouched.

Input p. The step adds $\{X? := T\}$ to σ_p^k, X the writer of the tail of p's person input stream or a writer in a message p put on its person output stream. In L_r the $\texttt{person}_p$ goal holds X: the input stream's tail from the initial goal, and a message's writer once a Communicate delivers the message to it, which $\hat{r}$ takes first where r has not. The clause of $\texttt{person}_p$ for this input has T at the position of X, so the Reduce of the $\texttt{person}_p$ goal with it has the writer mgu $\{X := T\}$, adding $\{X? := T\}$ to the substitution of $\hat{r}$, as in maGLP; r is finite, so $\texttt{person}_p$ has a clause per input.

Communicate p to q. Here $\{X? := T\} \in \sigma_p^k$ and $X?$ occurs in G_q^k. Both lie in the substitution and the goal of $\hat{r}$, so the cGLP Communicate applying $\{X? := T\}$ is a step of L_r; as $X?$ occurs once in the goal, it

instantiates exactly the occurrence in q's goal, as in maGLP. The maGLP step removes $\{X? := T\}$ from σ_p^k whereas the cGLP rule retains it, but $X?$ then occurs nowhere, so neither any subsequent goal nor the outcome is affected. When $p = q$ the transaction is unary and is itself a cGLP Communicate step of L_r.

Cold-call p to q. The network output stream of c_p^k has a new message $\texttt{msg}(q,X)$, and the step adds $\texttt{msg}(q,X)$ to the network input stream of c_q^k. In L_r the `network` goal consumes $\texttt{msg}(q,X)$ from p's network output stream and places $\texttt{msg}(q,X)$ on q's network input stream, routing by the network-switch technique, generalised to P. This is a bounded sequence of cGLP steps of L_r — a Communicate delivering $\texttt{msg}(q,X)$ to the `network` goal, the Reduce of the routing clause, and a Communicate placing $\texttt{msg}(q,X)$ on q's network input stream — after which $\texttt{msg}(q,X)$ occurs on that stream, as in maGLP.

Each maGLP step is simulated by cGLP transitions of L_r, so $\hat{r}$ is a run of L_r. Reduce introduces only variables renamed apart, Input only fresh variables, and Communicate none, so $\hat{r}$ inherits properness from r. By cGLP Computation is Deduction (Proposition 2.13) applied to L_r over M_r', the outcome of $\hat{r}$ is a logical consequence of $M_r'/?$. □

E.3 Grassroots Proofs

Proposition E.7 (Obliviousness Condition with Input)**.** *A transactions-based protocol with input over a contained set of transactions is oblivious provided that for every disjoint nonempty $P,P' \subset \Pi$, no class whose transactions have participants spanning both P and P' is ever enabled in any interleaving of correct runs of $\mathscr{F}^I(P)$ and $\mathscr{F}^I(P')$.*

Proof. The proof of the Obliviousness Condition (Proposition C.10) in [24] applies, with one addition to each of its two parts. Let e be an interleaving of a correct run r of $\mathscr{F}^I(P)$ and a correct run of $\mathscr{F}^I(P')$. *Safety*: a P-step of e by a transition of $I(P)\!\uparrow\!P$ is a transition of $I(P\cup P')\!\uparrow\!(P\cup P')$, since $I(P) \subseteq I(P\cup P')$ and the agents of P' are stationary in it; every other step is as there. *Liveness*: the argument there concerns the classes of $\mathscr{F}^I(P\cup P')$, which are those of $R(P\cup P')\!\uparrow\!(P\cup P')$, a transition of I being in none; and the runs r and r' it draws on are correct runs of $\mathscr{F}^I(P)$ and $\mathscr{F}^I(P')$, whose transitions of I owe nothing to liveness. □

Theorem E.8 (Transactions-Based Grassroots)**.** *A transactions-based protocol with input that satisfies the condition of Proposition C.12 and is interactive is grassroots.*

Proof. By Proposition C.12, the protocol is oblivious. If it is also interactive, it is grassroots by Definition C.6. □

Theorem E.9. *The maGLP protocol over a program that names no agent and uses cold calls is grassroots.*

Proof. Let M name no agent and use cold calls, and $\mathscr{F}$ be the maGLP protocol over M. By Theorem C.13, it suffices to show that $\mathscr{F}$ is a transactions-based protocol with input over a contained set of transactions, that it satisfies the condition of Proposition C.12, and that it is interactive.

Transactions-based with input: Let R be the Reduce, Communicate and Cold-call transactions of Definition 3.6 over every finite set of agents, I its Input transactions, and $\sim$ the maGLP transaction equivalence. Every participant of every transaction changes its state, so distinct transactions have disjoint closures (Section 3.1). A transaction among agents of P between configurations of $S(P)^P$ is over $S(P)$, so $\mathscr{F}(P)$, the maGLP transition system over P and M restricted to $S(P)^P$, is $(S(P)^P, \{s_0\}^P, R(P)\!\uparrow\!P \cup I(P)\!\uparrow\!P, \sim\!\uparrow\!P)$, and $\mathscr{F}$ is the protocol over R, S, $\sim$ and input I.

Contained: Let $t \in R$ have participants $Q \subseteq P$ and source in $S(P)^Q$. Every agent occurring in the target of t occurs in its source or in M: Reduce p replaces a goal by the body of a clause of M renamed apart, instantiated by a writer mgu whose terms are subterms of the goal and the clause; Communicate p to q moves a term of σ_p into G_q; Cold-call p to q moves $\mathtt{msg}(q,X)$ from p's network output stream to q's network input stream. As M names no agent, the target is in $S(P)^Q$, and $t \in R(P)$.

Obliviousness condition: A class whose transactions have participants spanning both P and P' is one of Communicate or Cold-call transactions between some $p \in P$ and $q \in P'$. In an interleaving of correct runs of $\mathscr{F}(P)$ and $\mathscr{F}(P')$, the state of p is a state of the run of $\mathscr{F}(P)$, in $S(P)$, so no message $\mathtt{msg}(q,X)$ is on p's network output stream, and Cold-call p to q is not enabled. Communicate p to q requires p and q to share a variable pair. The variables at the agents of P in a run of $\mathscr{F}(P)$ are in $\bigcup_{p\in P} \hat{\mathscr{V}}_p$: those of the initial state at p, those Reduce p renames into and those Input p writes are in $\hat{\mathscr{V}}_p$ (Definition 3.6), and Communicate and Cold-call move variables among the agents of P. Likewise for P', and the two unions are disjoint (Definition 3.5), so Communicate p to q is not enabled. The condition of Proposition C.12 is satisfied.

Interactivity: Let $Q \subset \Pi$, $P, P' \subseteq Q$ be disjoint and nonempty, $\hat{r}$ a prefix of a correct run of $\mathscr{F}(Q)$ containing no interaction between P and P', $p \in P$ and $q \in P'$. As M uses cold calls, $\hat{r}$ extends, by an Input p writing a term in which q occurs and by Reduce p and Communicate p to p transactions, to a finite safe run of $\mathscr{F}(Q)$ at whose last configuration Cold-call p to q is enabled; extend it by that Cold-call. The Cold-call changes the states of p and q, and p's step is not a transition of $\mathscr{F}(\{p\})$, its source naming $q \notin \{p\}$; so it is an interaction between p and q (Definition C.5), and hence between P and P'. The result is a finite safe run of $\mathscr{F}(Q)$ having $\hat{r}$ as a prefix and containing an interaction between P and P', and by Lemma C.3 it is a prefix of a correct run of $\mathscr{F}(Q)$. Hence $\mathscr{F}$ is interactive. □

Corollary E.10. *The GLP implementation of the grassroots social graph (Section 4) is grassroots.*

Proof. The social graph program names no agent: its constants are message tags, and an agent's own identity and its friends' reach it through the person channel and the network. It uses cold calls: in every state, the person's `connect(Target)` request naming an agent is followed by the reduction of the clause of Section 4 that sends $\mathtt{msg}(\mathtt{Target}, \mathtt{intro}(\mathtt{Id}, \mathtt{Resp}))$ on the network output stream, at which Cold-call to `Target` is enabled. By Theorem 5.3, it is grassroots. □

F Social Graph Walkthrough

This appendix presents the detailed walkthrough of the grassroots social graph program introduced in Section 4.

F.1 Types and Channels

The protocol type vocabulary — `Response`, `Decision`, `NetColdCall`, `FriendContent`, `IntroContent`, `OutputEntry`, `OutputsList`, and the rest — is defined in `self.glp` of the repository directory (Section 4).

A `Channel` is a pair of streams for bidirectional communication. The `new_channel` guard, shown in Section 4, creates complementary channel endpoints: each reads from its first stream and writes to its second, and each end's outgoing stream is the other end's incoming stream.

F.2 Agent Structure

Each agent processes messages from two separate input streams—user and network—and maintains a typed outputs list (Section 4). The four arguments of `agent/4` are: agent identity, user input

stream, network input stream, and the outputs list. The outputs list initially contains two entries—`user_output(...)` and `net_output(...)`—providing streams to the user interface and the network, respectively. As the agent befriends others, `friend_output` entries are added.

F.3 Cold-Call Befriending Protocol

The cold-call protocol enables agents to establish friendship without prior shared variables. Its initiating, receiving, and decision clauses appear in Section 4; the remaining clauses, among them the two that handle the response to a sent cold-call, are in `agent.glp` of the repository directory.

The protocol works as follows: (1) Alice sends `connect(bob)` to her agent via the `person` stream; (2) her agent sends a 2-argument `msg(bob, intro(alice, Resp))` via the `net_output` entry, including a fresh response variable `Resp`; (3) Bob receives `msg(bob, intro(alice, Resp))` on his network input and forwards the request to his user interface; (4) Bob decides `yes` or `no`; (5) Bob's agent creates a setup-channel pair and sends `accept(RetCh?)` back via `Resp`; (6) Alice's agent receives the response, and both agents commit the friendship via the shared `befriend_commit`, which conveys a fresh friend channel over the setup channel and adds a `friend_output` entry on each side.

F.4 Channel Establishment

When a cold-call is accepted, both agents establish symmetric channels; the procedures `bind_response` and `handle_response` are in `agent.glp` of the repository directory.

The accepting agent creates a setup-channel pair via `new_channel(RetCh, LocalCh)`, sends `RetCh` back to the initiator, and commits over `LocalCh`. The shared library procedure `befriend_commit` creates the fresh friend channel, conveys it as `friend_channel(...)` over the setup channel, adds the `friend_output` entry, and merges the friend's incoming stream into the agent's network input; it is idempotent, so both ends run the same commit.

F.5 Text Messaging

Once agents are friends, they can exchange text messages; the sending and receiving clauses appear in Section 4.

F.6 Friend-Mediated Introduction

Once agents are friends, they can introduce each other to third parties. The introducing clause, which creates a fresh channel pair and sends each half to the respective parties, appears in Section 4; the receiving-side clauses are in `agent.glp` of the repository directory, and the procedures they call, `intro_await_peer` and `inject_intro_result`, are in `routing/intro.glp` of the repository directory.

When Bob types `introduce(alice, charlie)`, he creates a setup-channel pair via `new_channel(PQCh, QPCh)`; Alice and Charlie each receive one half. Accepting sends `ack` on the setup channel; `intro_await_peer` waits for the peer's `ack` (or `nack`) and injects the result, and on mutual acknowledgement each side commits via `befriend_commit`. When both accept, they become direct friends without Bob's further involvement.

Note the distinction between cold-call messages and friend-mediated messages: cold-call uses a 2-argument `msg(Target, Content)` sent via the `net_output` entry, while friend-mediated introduction uses a 3-argument `msg(From, To, intro(Other, Ch))` sent directly to a friend via their named output.

F.7 Network Switch Simulation

In deployment, agents communicate through a physical network. In simulation, a `network3` process routes messages between three agents; it is listed in full in `boot.glp` of the repository directory (Section 4).

The network switch routes both 2-argument cold-call messages (where `msg(Target, Content)` is addressed by target name) and 3-argument friend-to-friend messages (where `msg(From, To, Content)` carries both sender and receiver identities). In deployment, the network switch is replaced by maGLP's Cold-call transaction (Section 3).

F.8 The Scenario and Actors

The complete scenario demonstrates three of the four protocols of Section 4; unfriending is not among them:

1. Alice cold-calls Bob (Bob accepts) — Alice and Bob become friends
2. Alice sends Bob: "Hi Bob, this is Alice"
3. Bob cold-calls Charlie (Charlie accepts) — Bob and Charlie become friends
4. Charlie sends Bob: "Hi Bob, this is Charlie"
5. Bob introduces Alice to Charlie (both accept) — Alice and Charlie become direct friends
6. Alice sends Charlie: "Hi Charlie, this is Alice"
7. Charlie responds: "Hi Alice, this is Charlie"

Each agent is driven by an *actor*—a GLP procedure that implements a state machine, reacting to messages from the UI mediator and producing commands. The actor communicates in ground terms only—the same protocol a human uses in the interactive UI. The actor's state is encoded in procedure names, with transitions via recursive calls. The complete program — helper procedures, the UI mediator, the actors, the plays that tie everything together, and the boot variants — is the repository directory `programs/social/graph` (Section 4).